



5장 회귀 – Kaggle 필사

3팀 김서연, 박린, 진웨이안

목차

#01 5-9장. 자전거 수요 예측 + Insurance Prediction with five Regressor Models

#02 5-10장. 캐글 주택 가격: 고급 회귀 기법

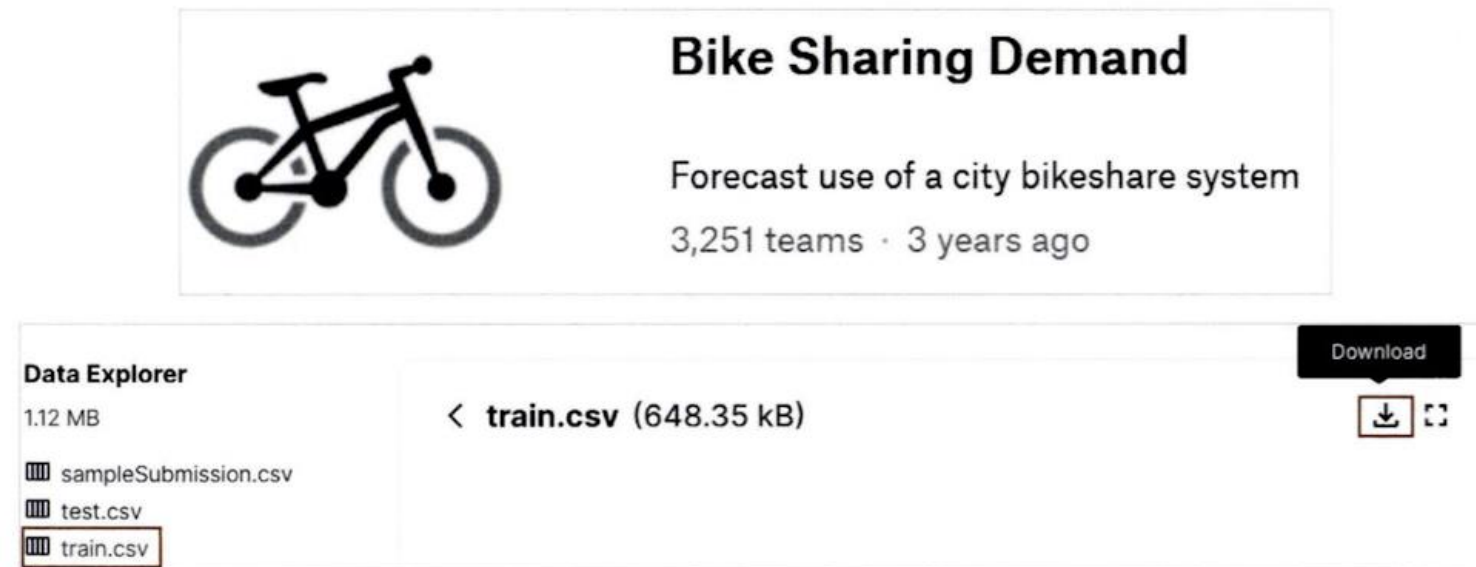
#03 Avocado Price Regression w/ PyCaret & EDA



5.9 자전거 대여 수요 예측



5.9 자전거 대여 수요 예측



- 2011년 1월부터 2012년 12월까지 날짜/시간, 기온, 습도, 풍속 등의 정보를 기반으로 1시간 간격 동안의 자전거 대여 횟수

- datetime: hourly date + timestamp
- season: 1 = 봄, 2 = 여름, 3 = 가을, 4 = 겨울
- holiday: 1 = 토, 일요일의 주말을 제외한 국경일 등의 휴일, 0 = 휴일이 아닌 날
- workingday: 1 = 토, 일요일의 주말 및 휴일이 아닌 주중, 0 = 주말 및 휴일
- weather:
 - 1 = 맑음, 약간 구름 낀 흐림
 - 2 = 안개, 안개 + 흐림
 - 3 = 가벼운 눈, 가벼운 비 + 천둥
 - 4 = 심한 눈/비, 천둥/번개
- temp: 온도(섭씨)
- atemp: 체감온도(섭씨)
- humidity: 상대습도
- windspeed: 풍속
- casual: 사전에 등록되지 않는 사용자가 대여한 횟수
- registered: 사전에 등록된 사용자가 대여한 횟수
- count: 대여 횟수

5.9.1 데이터 클렌징 및 가공

- 데이터 확인

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

import warnings
warnings.filterwarnings("ignore", category=RuntimeWarning)

from google.colab import drive
drive.mount('/content/drive')

bike_df = pd.read_csv('/content/drive/MyDrive/EuronData/bike_train.csv')
print(bike_df.shape)
bike_df.head(3)
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

(10886, 12)

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	27	32

5.9.1 데이터 클렌징 및 가공

- Null 데이터는 없음
- Datetime 칼럼만 object형
- 문자열을 판다스의 'datetime' 타입으로 변경

```
bike_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   datetime    10886 non-null  object
1   season      10886 non-null  int64
2   holiday     10886 non-null  int64
3   workingday  10886 non-null  int64
4   weather     10886 non-null  int64
5   temp        10886 non-null  float64
6   atemp       10886 non-null  float64
7   humidity    10886 non-null  int64
8   windspeed   10886 non-null  float64
9   casual      10886 non-null  int64
10  registered  10886 non-null  int64
11  count       10886 non-null  int64
dtypes: float64(3), int64(8), object(1)
memory usage: 1020.7+ KB
```

```
# 문자열을 datetime 타입으로 변경.
bike_df['datetime'] = bike_df.datetime.apply(pd.to_datetime)

# datetime 타입에서 년, 월, 일, 시간 추출
bike_df['year'] = bike_df.datetime.apply(lambda x : x.year)
bike_df['month'] = bike_df.datetime.apply(lambda x : x.month)
bike_df['day'] = bike_df.datetime.apply(lambda x : x.day)
bike_df['hour'] = bike_df.datetime.apply(lambda x : x.hour)
bike_df.head(3)
```

5.9.1 데이터 클렌징 및 가공

```
drop_columns = ['datetime', 'casual', 'registered']  
bike_df.drop(drop_columns, axis=1, inplace=True)
```

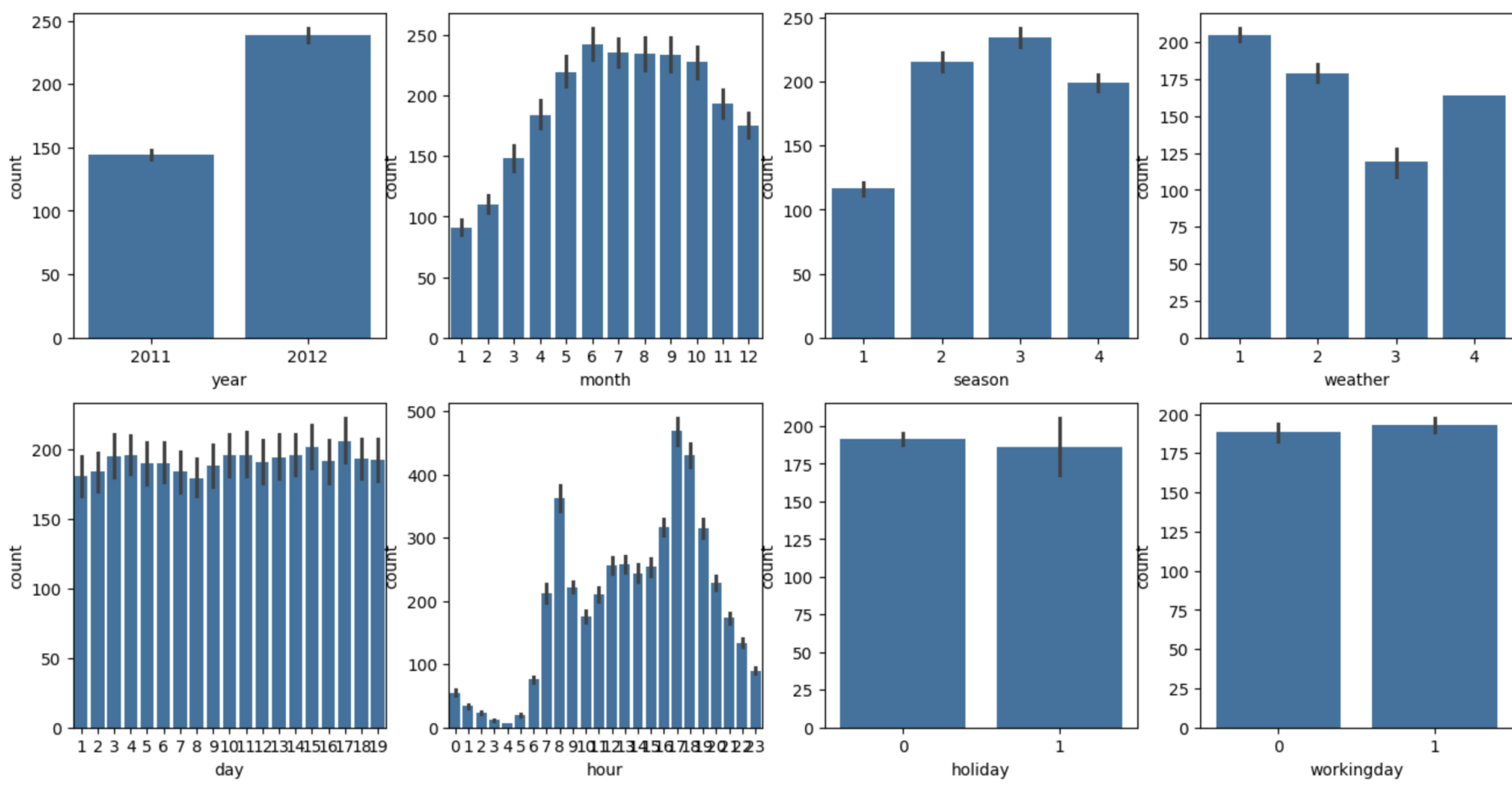
- 새롭게 year, month, day, hour 칼럼 추가
→ datetime 칼럼 삭제
- Casual + registered = count
→ casual과 registered 칼럼 삭제
→ 상관도가 높아 예측 저해 우려 있음

5.9.1 데이터 클렌징 및 가공

• 데이터 시각화

```
fig, axs = plt.subplots(figsize=(16, 8), ncols=4, nrows=2)
cat_features = ['year', 'month', 'season', 'weather', 'day', 'hour', 'holiday', 'workingday']
# cat_features에 있는 모든 칼럼별로 개별 칼럼값에 따른 count의 합을 barplot으로 시각화
for i, feature in enumerate(cat_features):
    row = int(i/4)
    col = i%4
    # 시본의 barplot을 이용해 칼럼값에 따른 count의 합을 표현
    sns.barplot(x=feature, y='count', data=bike_df, ax=axs[row][col])
```

- 년도: 2012 > 2011
→ 시간이 지날수록 대여 횟수 지속적 증가
- 월: 6, 7, 8, 9월 > 1, 2, 3월
- 계절: 여름, 가을 > 봄, 겨울
- 날씨: 맑음, 안개 > 눈, 비
- 시간: 오전 출근시간, 오후 퇴근 시간 ↑



5.9.1 데이터 클렌징 및 가공

- RMSLE, MAE, RMSE 한꺼번에 평가하는 성능 평가 함수
- RMSLE(Root Mean Square Log Error)
 - > 오류 값의 로그에 대한 RMSE
 - > 실제값과 예측값에 log 변환을 적용해 제곱 평균 오차의 루트를 계산
 - > 작은 값의 오차에 민감하며, log1p 사용으로 0 처리 가능
- RMSE: 평균 제곱 오차의 제곱근으로, 큰 오차에 민감한 지표
 - > mean_squared_error() 함수 활용
- MAE: 오차의 절댓값 평균
- evaluate_regr() 함수는 위 세 지표를 함께 계산하고 결과를 출력

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

# log 값 변환 시 NaN등의 이슈로 log() 가 아닌 log1p() 를 이용하여 RMSLE 계산
def rmsle(y, pred):
    log_y = np.log1p(y)
    log_pred = np.log1p(pred)
    squared_error = (log_y - log_pred) ** 2
    rmsle = np.sqrt(np.mean(squared_error))
    return rmsle

# 사이킷런의 mean_squared_error() 를 이용하여 RMSE 계산
def rmse(y, pred):
    return np.sqrt(mean_squared_error(y, pred))

# MAE, RMSE, RMSLE 를 모두 계산
def evaluate_regr(y, pred):
    rmsle_val = rmsle(y, pred)
    rmse_val = rmse(y, pred)
    # MAE 는 scikit learn의 mean_absolute_error() 로 계산
    mae_val = mean_absolute_error(y, pred)
    print('RMSLE: {0:.3f}, RMSE: {1:.3f}, MAE: {2:.3f}'.format(rmsle_val, rmse_val, mae_val))
```


5.9.2 로그 변환, 피쳐 인코딩, 모델 학습/예측/평가

- LinearRegression으로 회귀 예측
→ 예측 오류로서는 비교적 큰 값
- 가장 큰 오류값 5개: 546~568
→ Target 값 분포 왜곡되어 있는지 확인
→ 정규 분포 형태가 좋음

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso

y_target = bike_df['count']
X_features = bike_df.drop(['count'], axis=1, inplace=False)

X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.3, random_state=0)

lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
pred = lr_reg.predict(X_test)

evaluate_regr(y_test, pred)

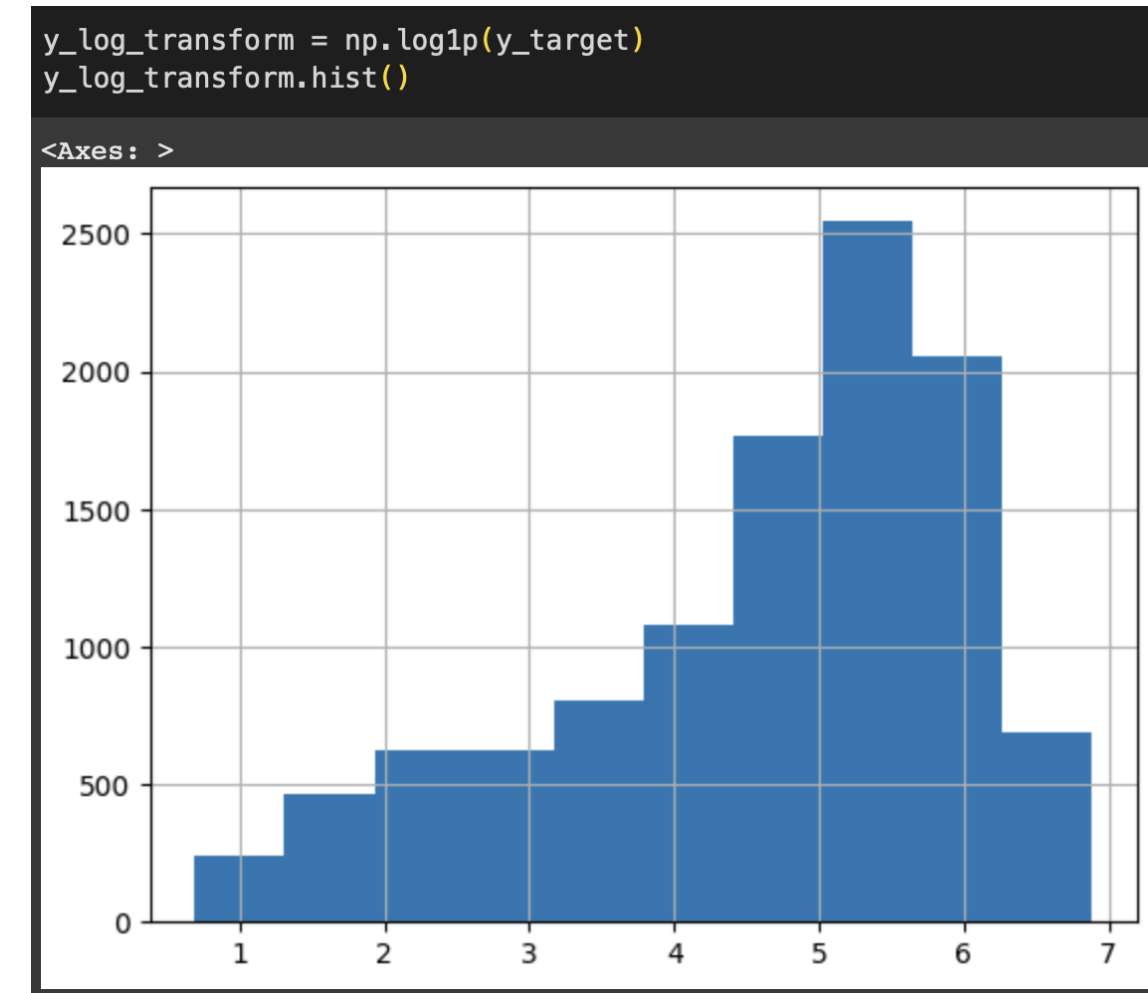
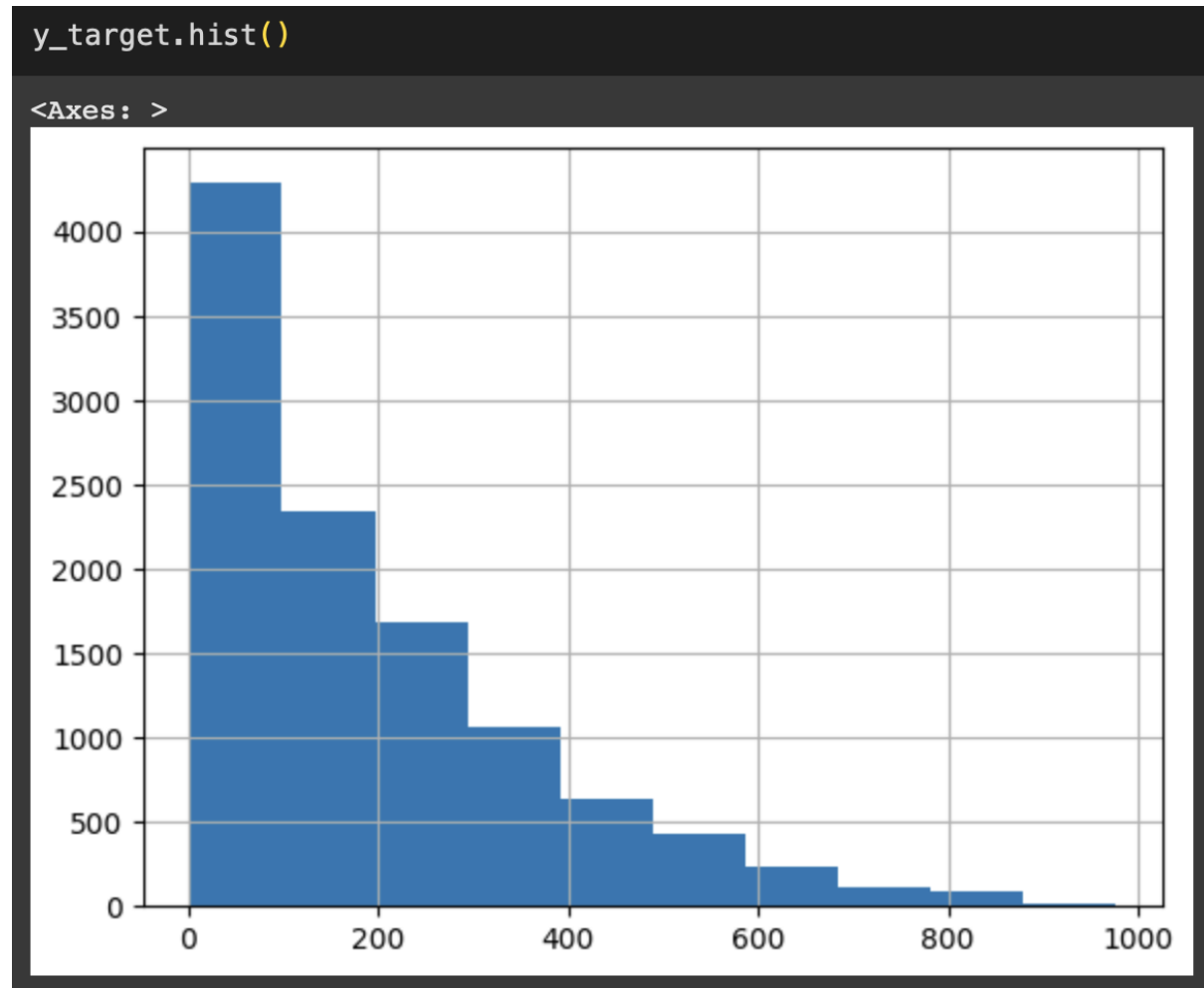
RMSLE: 1.165, RMSE: 140.900, MAE: 105.924
```

```
def get_top_error_data(y_test, pred, n_tops = 5):
    # DataFrame에 컬럼들로 실제 대여횟수(count)와 예측 값을 서로 비교 할 수 있도록 생성.
    result_df = pd.DataFrame(y_test.values, columns=['real_count'])
    result_df['predicted_count'] = np.round(pred)
    result_df['diff'] = np.abs(result_df['real_count'] - result_df['predicted_count'])
    # 예측값과 실제값이 가장 큰 데이터 순으로 출력.
    print(result_df.sort_values('diff', ascending=False)[:n_tops])

get_top_error_data(y_test, pred, n_tops=5)
```

	real_count	predicted_count	diff
1618	890	322.0	568.0
966	884	327.0	557.0
3151	798	241.0	557.0
412	745	194.0	551.0
2817	856	310.0	546.0

5.9.2 로그 변환, 피처 인코딩, 모델 학습/예측/평가



- Count 칼럼 값이 0~200 사이에 왜곡되어 있음
→ 로그 변환: $\log_{1p}()$ 후 학습, 예측한 값은
다시 $\expm1()$ 함수를 적용해 원상 복구

- $\log_{1p}()$ 왜곡 정도 많이 향상

5.9.2 로그 변환, 피쳐 인코딩, 모델 학습/예측/평가

```
# 타겟 칼럼인 count 값을 log1p로 로그 변환
y_target_log = np.log1p(y_target)

# 로그 변환된 y_target_log를 반영하여 학습/테스트 데이터 셋 분할
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target_log, test_size=0.3, random_state=0)
lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
pred = lr_reg.predict(X_test)

# 테스트 데이터 셋의 Target 값은 Log 변환되었으므로 다시 expm1를 이용하여 원래 scale로 변환
y_test_exp = np.expm1(y_test)

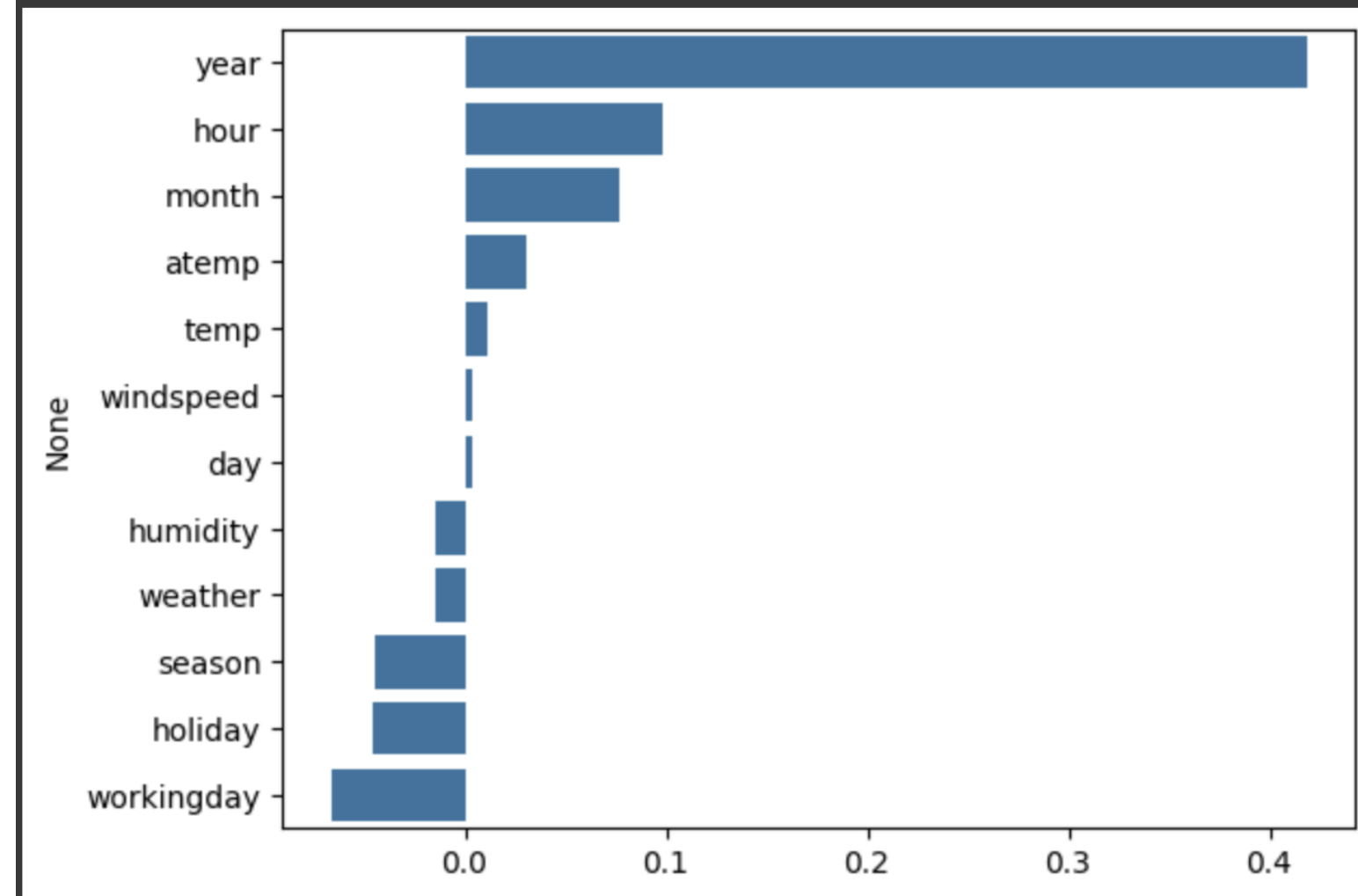
# 예측 값 역시 Log 변환된 타겟 기반으로 학습되어 예측되었으므로 다시 expm1으로 scale변환
pred_exp = np.expm1(pred)

evaluate_regr(y_test_exp, pred_exp)

RMSLE: 1.017, RMSE: 162.594, MAE: 109.286
```

- RMSLE 감소, RMSE 증가

```
coef = pd.Series(lr_reg.coef_, index=X_features.columns)
coef_sort = coef.sort_values(ascending=False)
sns.barplot(x=coef_sort.values, y=coef_sort.index)
plt.savefig('log_transform.tif', format='tif', dpi=300, bbox_inches='tight')
```



- 각 피쳐 회귀 계수 시각화
- 개별 숫자값의 크기가 의미가 없는 카테고리 값
→ 원-핫 인코딩을 적용해 변환

5.9.2 로그 변환, 피처 인코딩, 모델 학습/예측/평가

```
# 'year', 'month', 'day', 'hour'등의 피처들을 One Hot Encoding
X_features_ohe = pd.get_dummies(X_features, columns=['year', 'month', 'day', 'hour', 'holiday',
                                                    'workingday', 'season', 'weather'])

# 원-핫 인코딩이 적용된 feature 데이터 세트 기반으로 학습/예측 데이터 분할.
X_train, X_test, y_train, y_test = train_test_split(X_features_ohe, y_target_log,
                                                    test_size=0.3, random_state=0)

# 모델과 학습/테스트 데이터 셋을 입력하면 성능 평가 수치를 반환
def get_model_predict(model, X_train, X_test, y_train, y_test, is_exp1=False):
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    if is_exp1:
        y_test = np.exp1(y_test)
        pred = np.exp1(pred)
    print('###', model.__class__.__name__, '###')
    evaluate_regr(y_test, pred)
# end of function get_model_predict

# model 별로 평가 수행
lr_reg = LinearRegression()
ridge_reg = Ridge(alpha=10)
lasso_reg = Lasso(alpha=0.01)

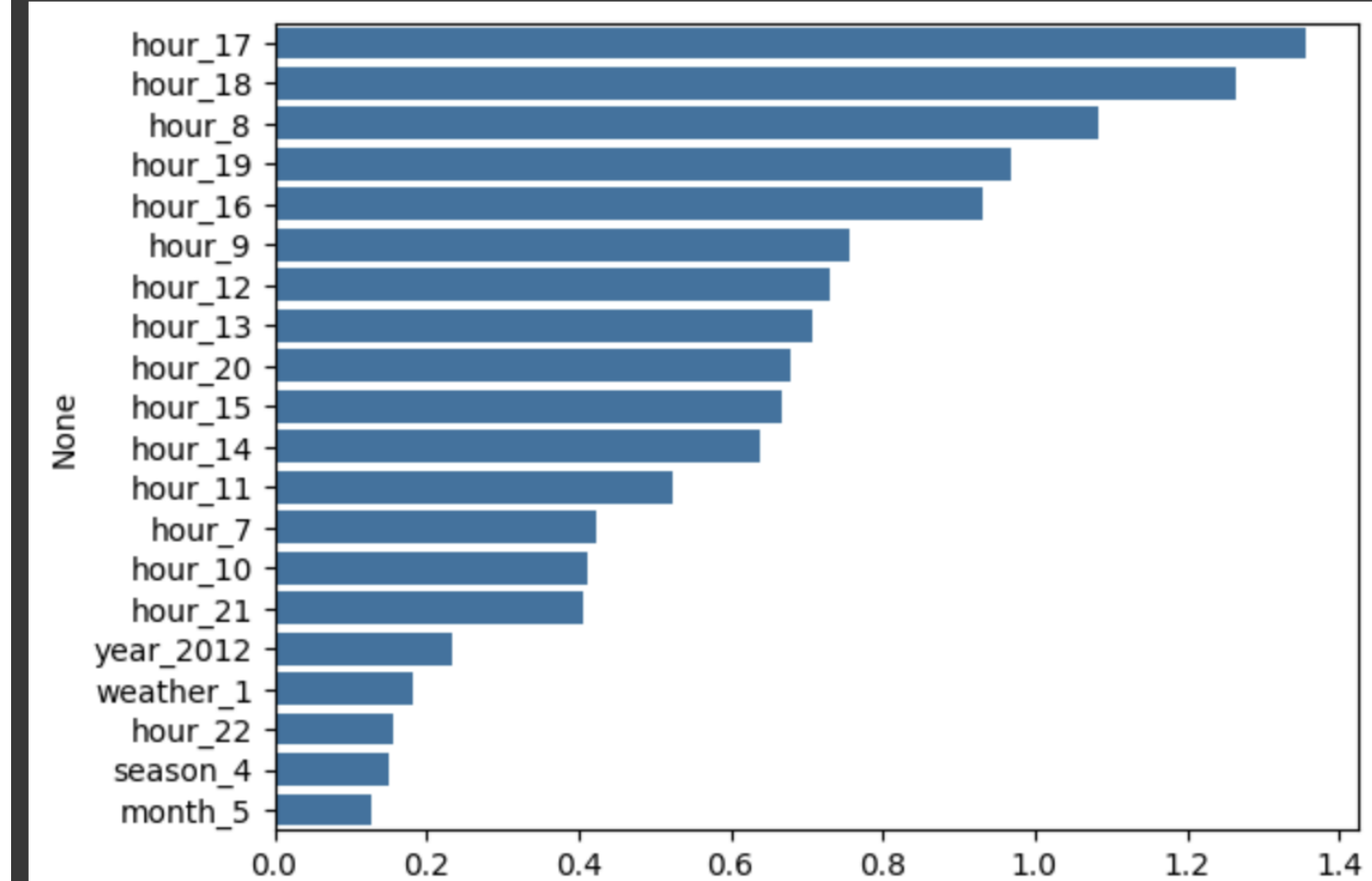
for model in [lr_reg, ridge_reg, lasso_reg]:
    get_model_predict(model, X_train, X_test, y_train, y_test, is_exp1=True)

### LinearRegression ###
RMSLE: 0.590, RMSE: 97.688, MAE: 63.382
### Ridge ###
RMSLE: 0.590, RMSE: 98.529, MAE: 63.893
### Lasso ###
RMSLE: 0.635, RMSE: 113.219, MAE: 72.803
```

- get_dummies() 이용해 원-핫 인코딩
- get_model_predict 함수로 성능 평가

```
coef = pd.Series(lr_reg.coef_, index=X_features_ohe.columns)
coef_sort = coef.sort_values(ascending=False)[:20]
sns.barplot(x=coef_sort.values, y=coef_sort.index)
```

<Axes: ylabel='None'>



- 피처들의 영향도가 달라지고 모델 성능 향상

5.9.2 로그 변환, 피처 인코딩, 모델 학습/예측/평가

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

# 랜덤 포레스트, GBM, XGBoost, LightGBM model 별로 평가 수행
rf_reg = RandomForestRegressor(n_estimators=500)
gbm_reg = GradientBoostingRegressor(n_estimators=500)
xgb_reg = XGBRegressor(n_estimators=500)
lgbm_reg = LGBMRegressor(n_estimators=500)

for model in [rf_reg, gbm_reg, xgb_reg, lgbm_reg]:
    # XGBoost의 경우 DataFrame이 입력 될 경우 버전에 따라 오류 발생 가능. ndarray로 변환.
    get_model_predict(model, X_train.values, X_test.values, y_train.values, y_test.values, is_exp1=True)

### RandomForestRegressor ###
RMSLE: 0.355, RMSE: 50.278, MAE: 31.128
### GradientBoostingRegressor ###
RMSLE: 0.330, RMSE: 53.344, MAE: 32.751
### XGBRegressor ###
RMSLE: 0.339, RMSE: 51.475, MAE: 31.357
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.001355 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 348
[LightGBM] [Info] Number of data points in the train set: 7620, number of used features: 72
[LightGBM] [Info] Start training from score 4.582043
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_fini
warnings.warn(
### LGBMRegressor ###
RMSLE: 0.319, RMSE: 47.215, MAE: 29.029
```

- 회귀 트리를 이용해 랜덤 포레스트, GBM, XGBoost, LightGBM 성능 평가
- 선형 회귀보다 성능 향상
but, 데이터 세트에 따라 결과 달라질 수 있음

Kaggle: Insurance prediction with five Regressor Models



Insurance prediction_데이터 준비 & 파악

- 데이터 파악

df.head(10)

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
5	31	female	25.740	0	no	southeast	3756.62160
6	46	female	33.440	1	no	southeast	8240.58960
7	37	female	27.740	3	no	northwest	7281.50560
8	37	male	29.830	2	no	northeast	6406.41070
9	60	female	25.840	0	no	northwest	28923.13692

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
1   sex         1338 non-null   object
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   object
5   region      1338 non-null   object
6   charges     1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

Insurance prediction_데이터 전처리

- 결측치 확인 -> 결측치 없음

```
Checking the missing data
```

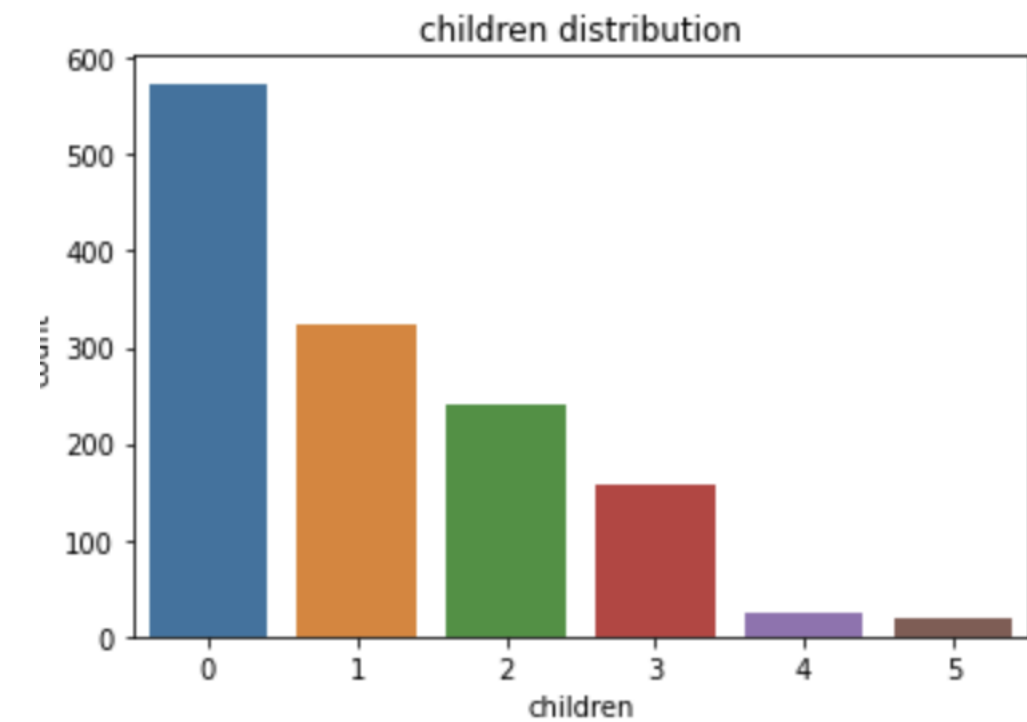
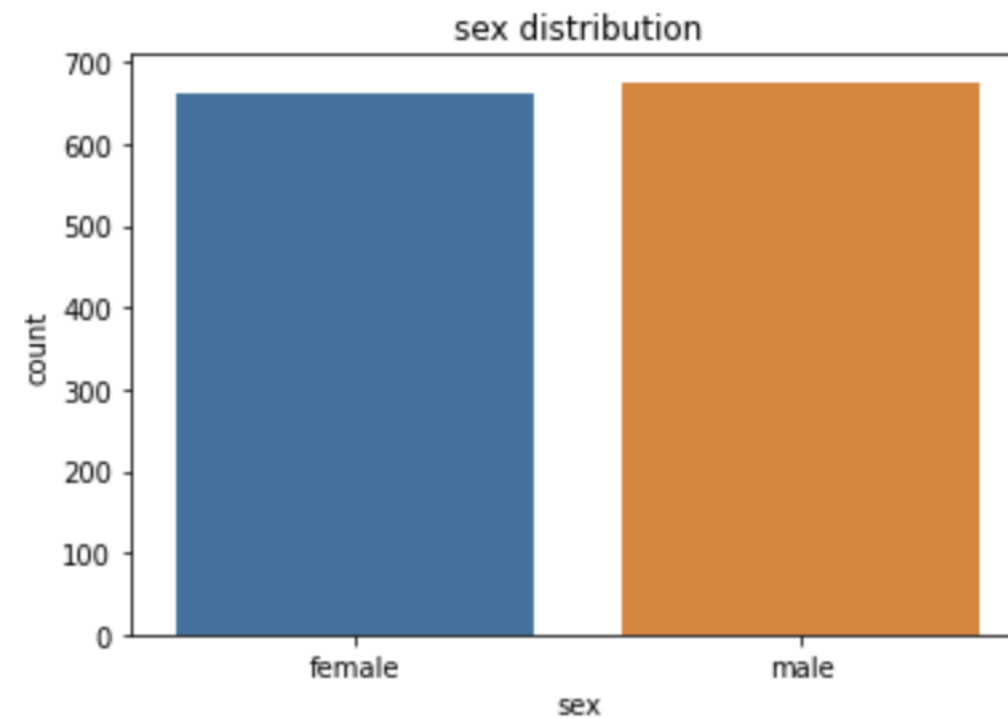
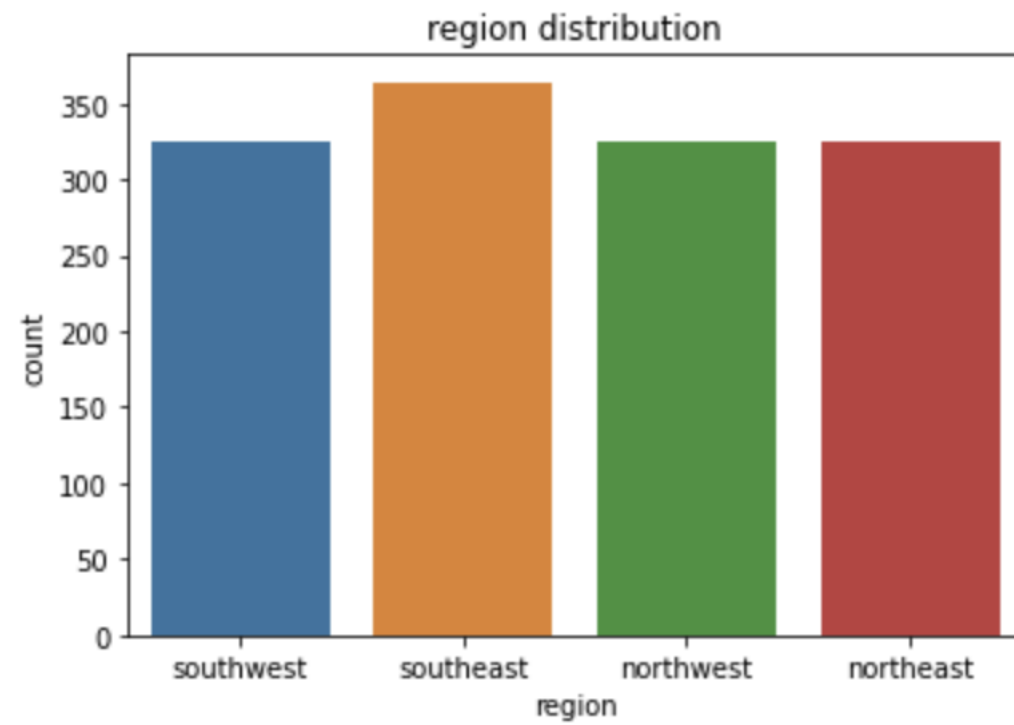
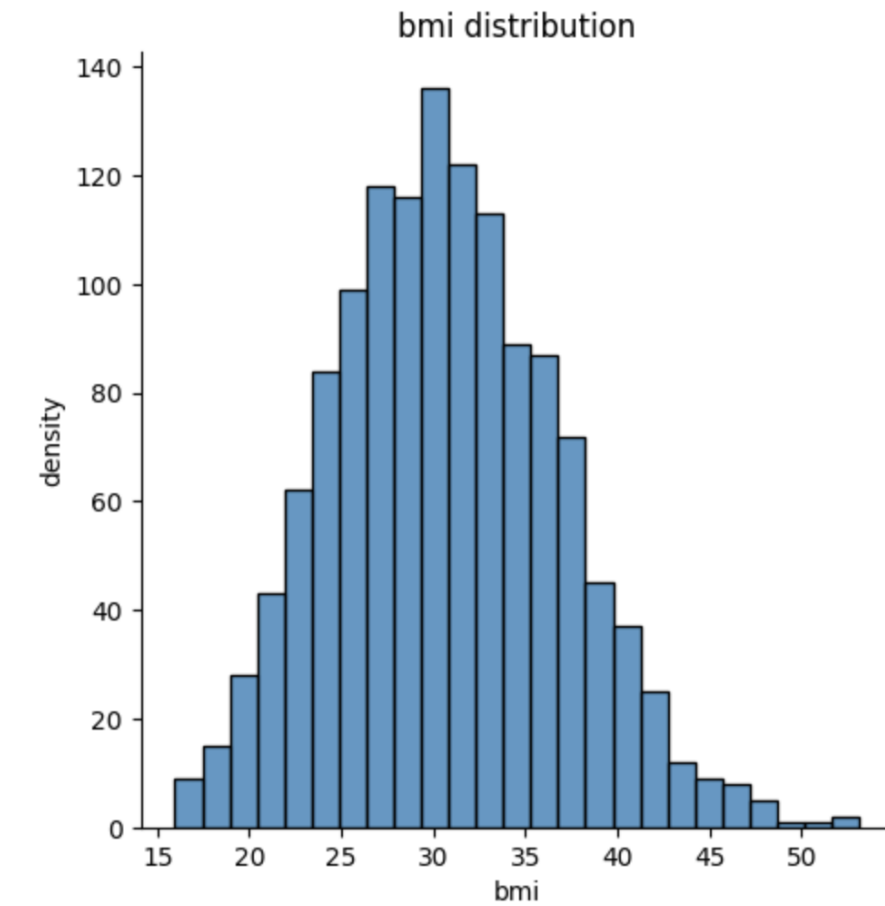
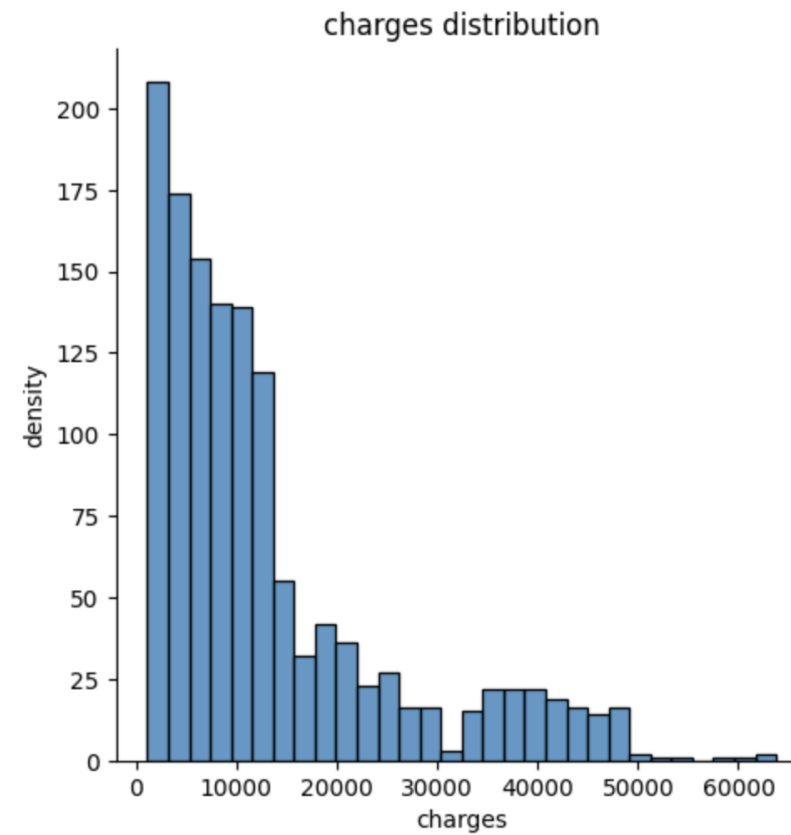
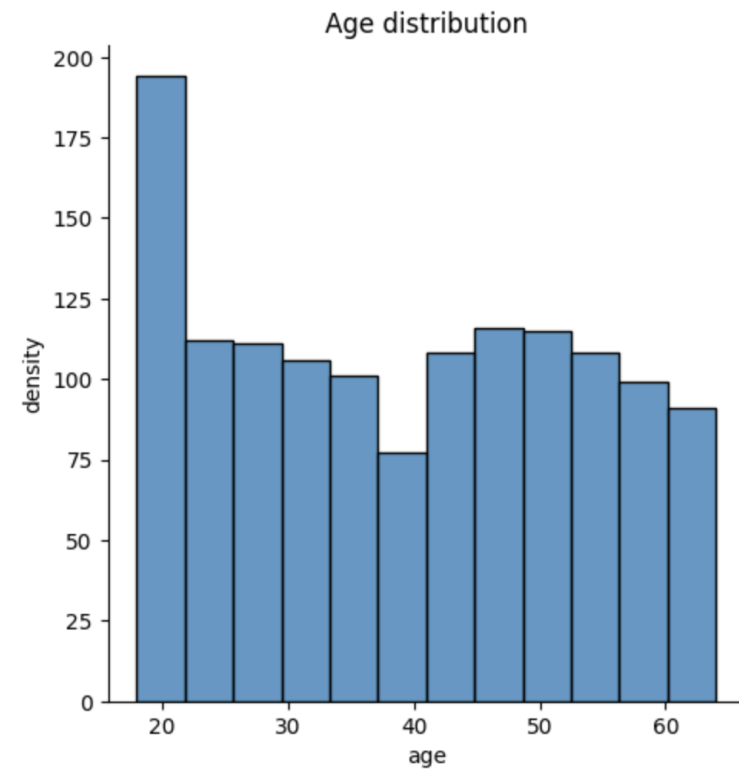
```
df.isnull().sum().sort_values(ascending=False)
```

	0
age	0
sex	0
bmi	0
children	0
smoker	0
region	0
charges	0

dtype: int64

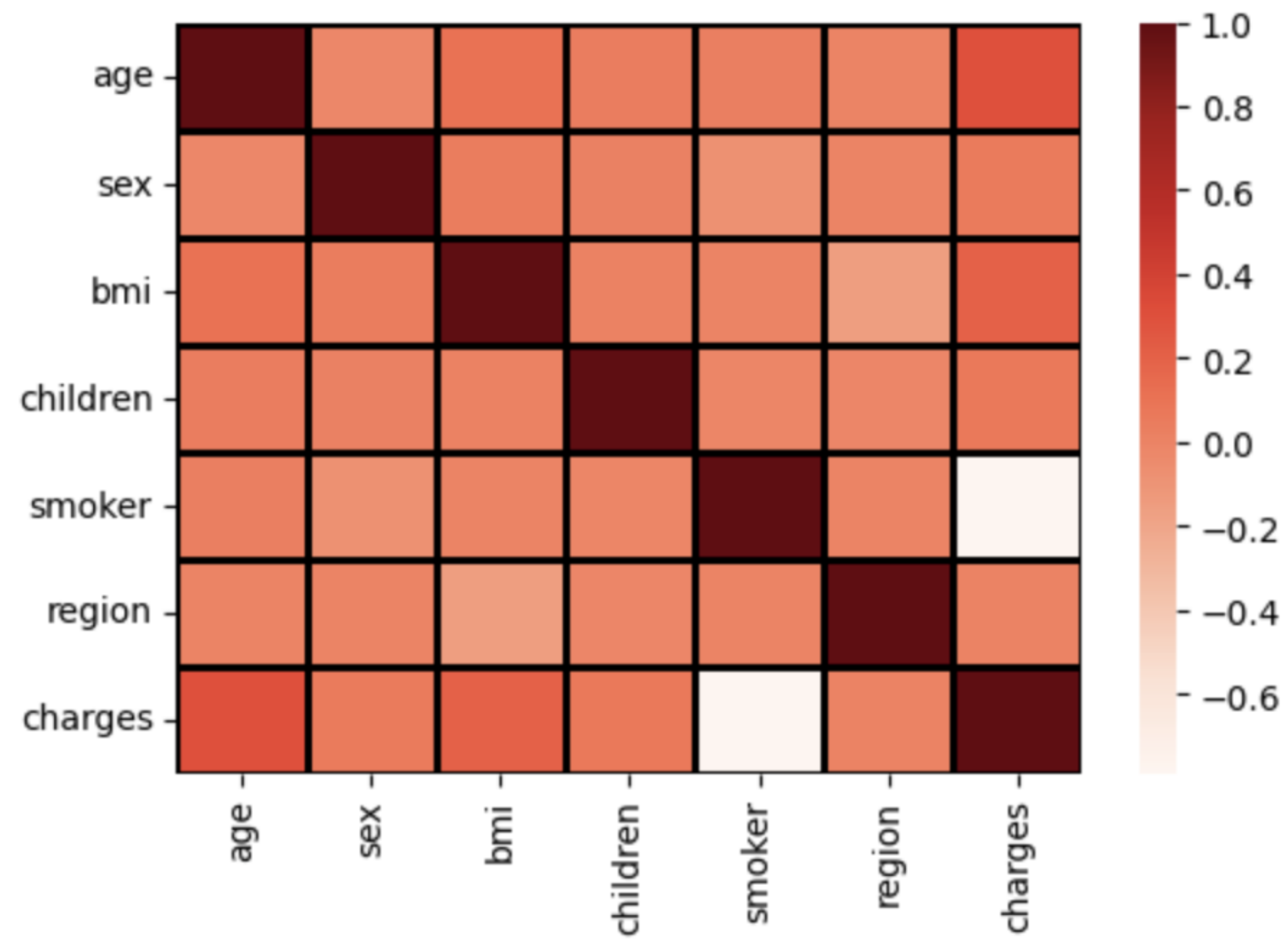
Insurance prediction_데이터 전처리

- 데이터 분포 시각화

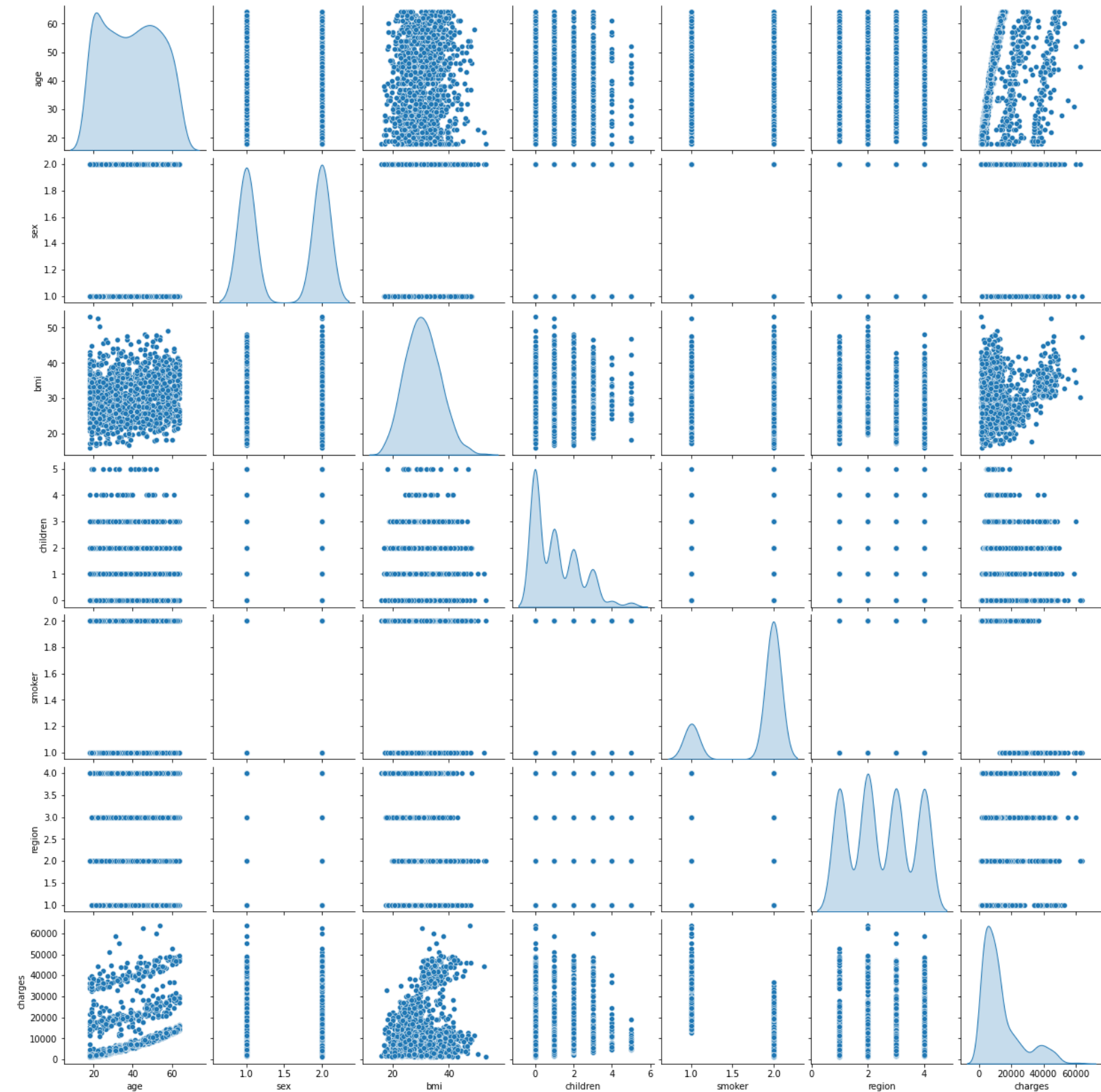


Insurance prediction_데이터 전처리

- 상관계수 확인



- 데이터프레임의 여러 변수 간의 관계를 한눈에 시각화



Insurance prediction_데이터 전처리

- Train_test_split

```
X = df.drop('charges', axis = 1)
y = df['charges']
X_train, X_test, y_train, y_test= train_test_split(X, y, test_size=0.3, random_state=101)
```

- Scaling

```
scaler= StandardScaler()
scaler.fit(X_train)
X_train_scaled= scaler.transform(X_train)
X_test_scaled= scaler.transform(X_test)
```

- StandardScaler()를 이용
-> 변수들의 단위 차이를 없애기 위해
평균 0, 표준편차 1로 변환

Insurance prediction_모델링

• Linear Regression Model

```
linear_reg_model= LinearRegression()
linear_reg_model.fit(X_train_scaled, y_train)
```

LinearRegression

```
LinearRegression()
```

```
[31] y_pred = linear_reg_model.predict(X_test_scaled)
y_pred = pd.DataFrame(y_pred)
MAE_li_reg= metrics.mean_absolute_error(y_test, y_pred)
MSE_li_reg = metrics.mean_squared_error(y_test, y_pred)
RMSE_li_reg =np.sqrt(MSE_li_reg)
pd.DataFrame([MAE_li_reg, MSE_li_reg, RMSE_li_reg], index=['MAE_li_reg', 'MSE_li_reg', 'RMSE_li_reg'], columns=['Metrics'])
```

	Metrics
MAE_li_reg	3.990250e+03
MSE_li_reg	3.353013e+07
RMSE_li_reg	5.790521e+03

```
[32] scores = cross_val_score(linear_reg_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))
```

```
[0.88791389 0.85653048 0.84404195 0.87198372 0.84417492]
```

```
r2_score(y_test, linear_reg_model.predict(X_test_scaled))
```

```
0.7613126015198817
```

- MAE, MSE, RMSE로 예측 성능 평가
- Scores: 5-fold 교차검증을 통해 성능 평가
- r2_score: 테스트 데이터를 이용한 단일 결정 계수
-> 1에 가까울수록 성능 좋음, 0이 평균

Insurance prediction_모델링

- Gradient Boosting Regressor Model

```
Gradient_model = GradientBoostingRegressor()
Gradient_model.fit(X_train_scaled, y_train)
```

▼ GradientBoostingRegressor

```
GradientBoostingRegressor()
```

```
[35] y_pred = Gradient_model.predict(X_test_scaled)
y_pred = pd.DataFrame(y_pred)
MAE_gradient = metrics.mean_absolute_error(y_test, y_pred)
MSE_gradient = metrics.mean_squared_error(y_test, y_pred)
RMSE_gradient = np.sqrt(MSE_gradient)
pd.DataFrame([MAE_gradient, MSE_gradient, RMSE_gradient], index=['MAE_gradient', 'MSE_gradient', 'RMSE_gradient'], columns=['Metrics'])
```

Metrics	
MAE_gradient	2.528851e+03
MSE_gradient	2.110725e+07
RMSE_gradient	4.594262e+03

```
scores = cross_val_score(Gradient_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))
```

```
[0.94585345 0.91473168 0.91679703 0.92149139 0.91695597]
```

```
r2_score(y_test, Gradient_model.predict(X_test_scaled))
```

```
0.849746079970233
```

- MAE, MSE, RMSE로 예측 성능 평가
- Scores: 5-fold 교차검증을 통해 성능 평가
- r2_score: 테스트 데이터를 이용한 단일 결정 계수
→ 1에 가까울수록 성능 좋음, 0이 평균

Insurance prediction_모델링

• XGB Regressor Model

```
[38] XGB_model =XGBRegressor()  
XGB_model.fit(X_train_scaled, y_train);  
  
[39] y_pred = XGB_model.predict(X_test_scaled)  
y_pred = pd.DataFrame(y_pred)  
MAE_XGB= metrics.mean_absolute_error(y_test, y_pred)  
MSE_XGB = metrics.mean_squared_error(y_test, y_pred)  
RMSE_XGB =np.sqrt(MSE_XGB)  
pd.DataFrame([MAE_XGB, MSE_XGB, RMSE_XGB], index=['MAE_XGB', 'MSE_XGB', 'RMSE_XGB'], columns=['Metrics'])  
  
Metrics  
MAE_XGB 3.142590e+03  
MSE_XGB 2.976529e+07  
RMSE_XGB 5.455757e+03  
  
[40] scores = cross_val_score(XGB_model, X_train_scaled, y_train, cv=5)  
print(np.sqrt(scores))  
  
[0.91782241 0.88730325 0.90410079 0.90440364 0.88383429]  
  
[41] r2_score(y_test, XGB_model.predict(X_test_scaled))  
  
0.7881129907007072
```

- MAE, MSE, RMSE로 예측 성능 평가
- Scores: 5-fold 교차검증을 통해 성능 평가
- r2_score: 테스트 데이터를 이용한 단일 결정 계수
→ 1에 가까울수록 성능 좋음, 0이 평균

Insurance prediction_모델링

• Decision Tree Regressor Model

```
[42] tree_reg_model = DecisionTreeRegressor()
tree_reg_model.fit(X_train_scaled, y_train);

[43] y_pred = tree_reg_model.predict(X_test_scaled)
y_pred = pd.DataFrame(y_pred)
MAE_tree_reg = metrics.mean_absolute_error(y_test, y_pred)
MSE_tree_reg = metrics.mean_squared_error(y_test, y_pred)
RMSE_tree_reg = np.sqrt(MSE_tree_reg)
pd.DataFrame([MAE_tree_reg, MSE_tree_reg, RMSE_tree_reg], index=['MAE_tree_reg', 'MSE_tree_reg', 'RMSE_tree_reg'], columns=['Metrics'])
```

Metrics	
MAE_tree_reg	3.412571e+03
MSE_tree_reg	4.821094e+07
RMSE_tree_reg	6.943410e+03

```
[44] scores = cross_val_score(tree_reg_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))

[0.8554894 0.8059276 0.86702127 0.86513289 0.84671168]
```

```
r2_score(y_test, tree_reg_model.predict(X_test_scaled))

0.6568058473191931
```

- MAE, MSE, RMSE로 예측 성능 평가
- Scores: 5-fold 교차검증을 통해 성능 평가
- r2_score: 테스트 데이터를 이용한 단일 결정 계수
→ 1에 가까울수록 성능 좋음, 0이 평균

Insurance prediction_모델링

- Random Forest Regressor Model

```
forest_reg_model = RandomForestRegressor()
forest_reg_model.fit(X_train_scaled, y_train);

[47] y_pred = forest_reg_model.predict(X_test_scaled)
y_pred = pd.DataFrame(y_pred)
MAE_forest_reg = metrics.mean_absolute_error(y_test, y_pred)
MSE_forest_reg = metrics.mean_squared_error(y_test, y_pred)
RMSE_forest_reg = np.sqrt(MSE_forest_reg)
pd.DataFrame([MAE_forest_reg, MSE_forest_reg, RMSE_forest_reg], index=['MAE_forest_reg', 'MSE_forest_reg', 'RMSE_forest_reg'], columns=['Metrics'])
```

	Metrics
MAE_forest_reg	2.847412e+03
MSE_forest_reg	2.441852e+07
RMSE_forest_reg	4.941510e+03

```
[48] scores = cross_val_score(forest_reg_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))

[0.93941297 0.90429466 0.92266941 0.91515676 0.90726894]

[49] r2_score(y_test, forest_reg_model.predict(X_test_scaled))

0.8261744658047943
```

- MAE, MSE, RMSE로 예측 성능 평가
- Scores: 5-fold 교차검증을 통해 성능 평가
- r2_score: 테스트 데이터를 이용한 단일 결정 계수
→ 1에 가까울수록 성능 좋음, 0이 평균

Insurance prediction_모델링

- Gradient Boosting Regressor Model

```
Gradient_model = GradientBoostingRegressor()
Gradient_model.fit(X_train_scaled, y_train)
```

▼ GradientBoostingRegressor ⓘ ⓘ

```
GradientBoostingRegressor()
```

```
[35] y_pred = Gradient_model.predict(X_test_scaled)
      y_pred = pd.DataFrame(y_pred)
      MAE_gradient= metrics.mean_absolute_error(y_test, y_pred)
      MSE_gradient = metrics.mean_squared_error(y_test, y_pred)
      RMSE_gradient = np.sqrt(MSE_gradient)
      pd.DataFrame([MAE_gradient, MSE_gradient, RMSE_gradient], index=['MAE_gradient', 'MSE_gradient', 'RMSE_gradient'], columns=['Metrics'])
```

	Metrics
MAE_gradient	2.528851e+03
MSE_gradient	2.110725e+07
RMSE_gradient	4.594262e+03

```
scores = cross_val_score(Gradient_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))
```

```
[0.94585345 0.91473168 0.91679703 0.92149139 0.91695597]
```

```
r2_score(y_test, Gradient_model.predict(X_test_scaled))
```

```
0.849746079970233
```

- 5가지 모델 중 Gradient Boosting Regressor Model의 예측 성능이 가장 좋게 나옴!

5-10 캐글 주택 가격: 고급 회귀 기법



데이터 사전 처리

```
import warnings
warnings.filterwarnings('ignore')
✓ import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

house_df_org = pd.read_csv(r"C:\Users\lynn\Documents\my_data\house_price.csv")
house_df = house_df_org.copy()
house_df.head(3)
```

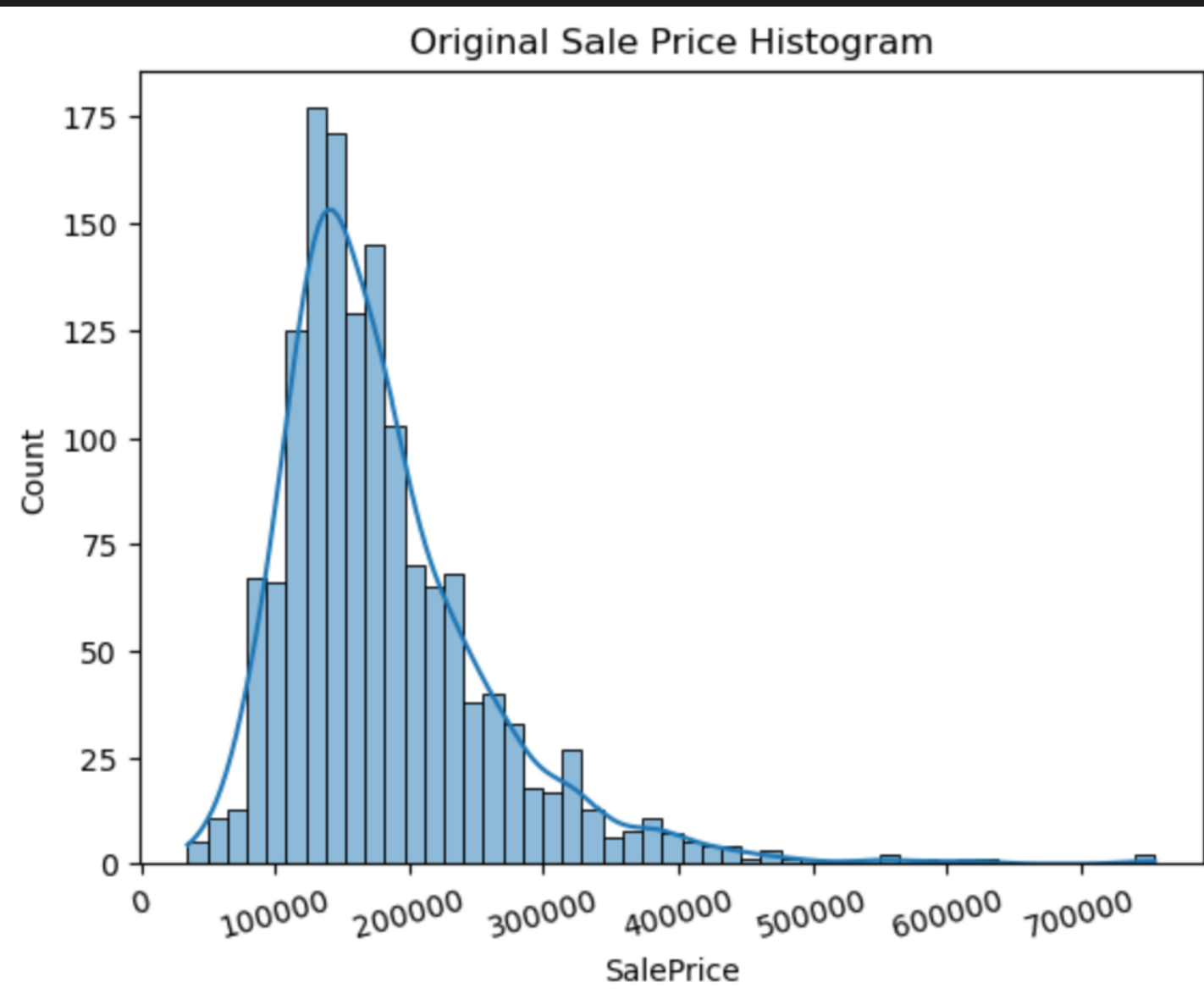
✓ 0.0s

데이터 사전 처리

#결과값 정규분포 형태로 변환

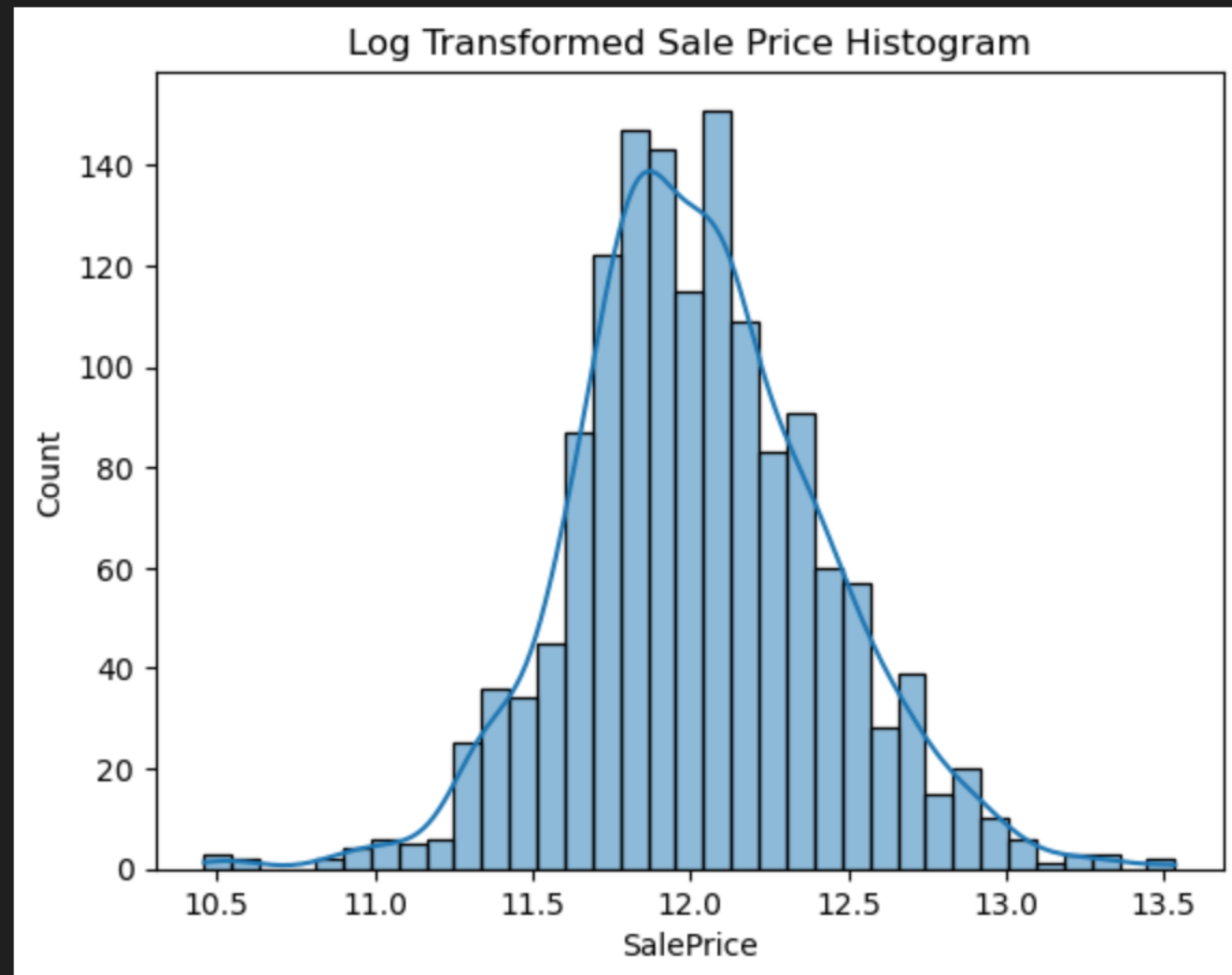
```
plt.title('Original Sale Price Histogram')
plt.xticks(rotation=15)
sns.histplot(house_df['SalePrice'], kde=True)
plt.show()
```

✓ 0.4s



```
plt.title('Log Transformed Sale Price Histogram')
log_SalePrice = np.log1p(house_df['SalePrice'])
sns.histplot(log_SalePrice, kde=True)
plt.show()
```

✓ 0.2s



데이터 사전 처리

#NULL값 처리

```
# SalePrice 로그 변환
original_SalePrice = house_df['SalePrice']
house_df['SalePrice'] = np.log1p(house_df['SalePrice'])

# Null 이 너무 많은 컬럼들과 불필요한 컬럼 삭제
house_df.drop(['Id', 'PoolQC', 'MiscFeature', 'Alley', 'Fence', 'FireplaceQu'], axis=1, inplace=True, errors='ignore')
# Drop 하지 않는 숫자형 Null컬럼들은 평균값으로 대체
house_df.fillna(house_df.mean(numeric_only=True), inplace=True)

# Null 값이 있는 피처명과 타입을 추출
null_column_count = house_df.isnull().sum()[house_df.isnull().sum() > 0]
print('## Null 피처의 Type :\n', house_df.dtypes[null_column_count.index])
```

✓ 0.0s

선형 회귀 모델 학습/예측/평가

#RMSE 측정을 위한 계산 함수 생성

```
def get_rmse(model):  
    pred = model.predict(X_test)  
    mse = mean_squared_error(y_test, pred)  
    rmse = np.sqrt(mse)  
    print('{0} 로그 변환된 RMSE: {1}'.format(model.__class__.__name__, np.round(rmse, 3)))  
    return rmse  
  
def get_rmses(models):  
    rmses = [ ]  
    for model in models:  
        rmse = get_rmse(model)  
        rmses.append(rmse)  
    return rmses
```

✓ 0.0s

선형 회귀 모델 학습/예측/평가

#선형 회귀 모델 학습, 예측, 평가

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

y_target = house_df_ohe['SalePrice']
X_features = house_df_ohe.drop('SalePrice', axis=1, inplace=False)
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.2, random_state=156)

# LinearRegression, Ridge, Lasso 학습, 예측, 평가
lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)

ridge_reg = Ridge()
ridge_reg.fit(X_train, y_train)

lasso_reg = Lasso()
lasso_reg.fit(X_train, y_train)

models = [lr_reg, ridge_reg, lasso_reg]
get_rmse(models)
```

✓ 0.0s

LinearRegression 로그 변환된 RMSE: 0.132

Ridge 로그 변환된 RMSE: 0.128

Lasso 로그 변환된 RMSE: 0.176

선형 회귀 모델 학습/예측/평가

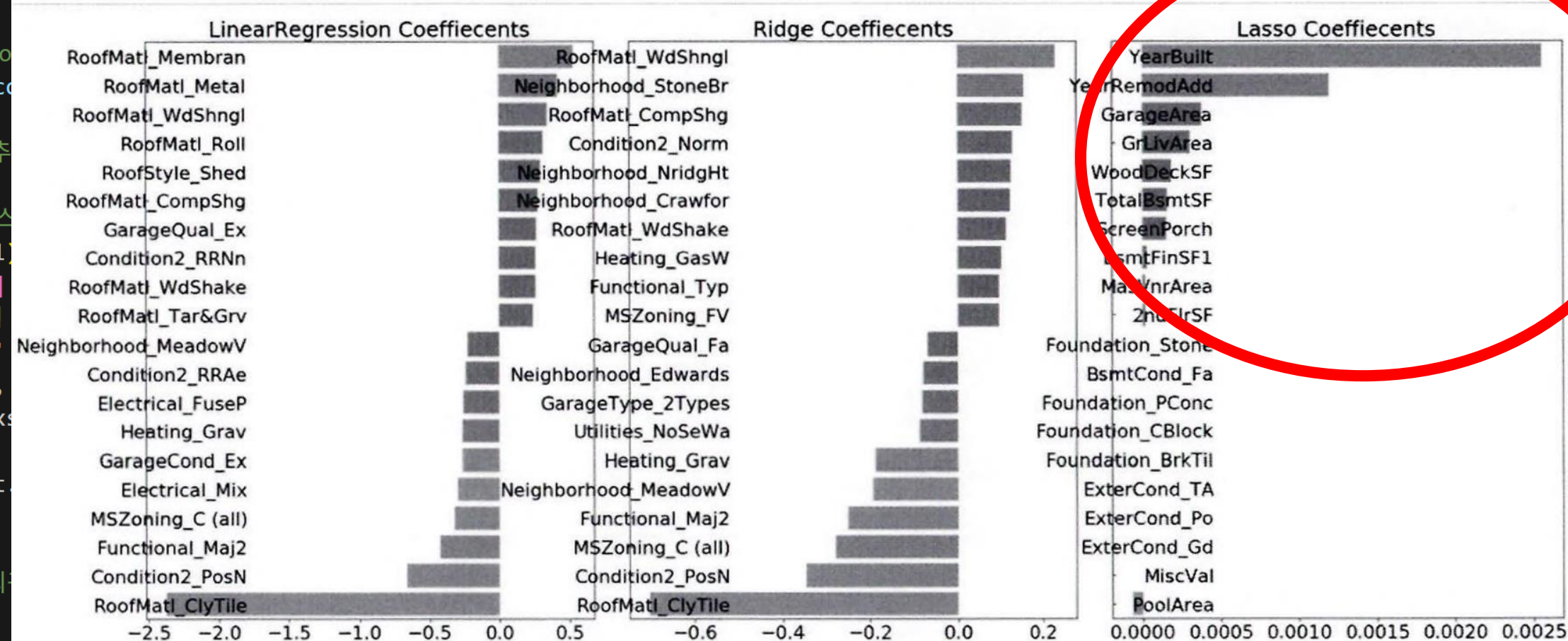
#선형 회귀 모델 학습, 예측, 평가

```
def get_top_bottom_coef(model):  
    # coef_ 속성을 기반으로 Series 객체를 생성. index는 컬럼명.  
    coef = pd.Series(model.coef_, index=X_features.columns)  
  
    # + 상위 10개 , - 하위 10개 coefficient 추출하여 반환.  
    coef_high = coef.sort_values(ascending=False).head(10)  
    coef_low = coef.sort_values(ascending=False).tail(10)  
    return coef_high, coef_low
```

✓ 0.0s

```
def visualize_coefficient(models):  
    # 3개 회귀 모델의 시각화를 위해 3개의 컬럼을 가지는 subplot  
    fig, axs = plt.subplots(figsize=(24,10),nrows=1,ncols=3)  
    fig.tight_layout()  
    # 입력인자로 받은 list객체인 models에서 차례로 model을 추  
    for i_num, model in enumerate(models):  
        # 상위 10개, 하위 10개 회귀 계수를 구하고, 이를 판다스  
        coef_high, coef_low = get_top_bottom_coef(model)  
        coef_concat = pd.concat([coef_high, coef_low])  
        # 순차적으로 ax subplot에 barchar로 표현. 한 화면에  
        axs[i_num].set_title(model.__class__.__name__ + '  
        axs[i_num].tick_params(axis="y",direction="in",  
        for label in (axs[i_num].get_xticklabels() + axs  
            label.set_fontsize(22)  
        sns.barplot(x=coef_concat.values, y=coef_concat  
        plt.show()  
  
    # 앞 예제에서 학습한 lr_reg, ridge_reg, lasso_reg 모델의 회  
    models = [lr_reg, ridge_reg, lasso_reg]  
    visualize_coefficient(models)
```

LinearRegression, Ridge와 상이한 형태
전체적으로 작은 회귀 계수 값



선형 회귀 모델 학습/예측/평가

#하이퍼 파라미터 튜닝

```
from sklearn.model_selection import GridSearchCV

def print_best_params(model, params):
    grid_model = GridSearchCV(model, param_grid=params,
                               scoring='neg_mean_squared_error', cv=5)
    grid_model.fit(X_features, y_target)
    rmse = np.sqrt(-1* grid_model.best_score_)
    print('{0} 5 CV 시 최적 평균 RMSE 값: {1}, 최적 alpha:{2}'.format(model.__class__.__name__,
                               np.round(rmse, 4), grid_model.best_params_))
    return grid_model.best_estimator_

ridge_params = { 'alpha':[0.05, 0.1, 1, 5, 8, 10, 12, 15, 20] }
lasso_params = { 'alpha':[0.001, 0.005, 0.008, 0.05, 0.03, 0.1, 0.5, 1, 5, 10] }
best_ribe = print_best_params(ridge_reg, ridge_params)
best_lasso = print_best_params(lasso_reg, lasso_params)
```

✓ 2.1s

Ridge 5 CV 시 최적 평균 RMSE 값:0.1418, 최적 alpha:{'alpha': 12}

Lasso 5 CV 시 최적 평균 RMSE 값:0.142, 최적 alpha:{'alpha': 0.001}

0.176 ->

선형 회귀 모델 학습/예측/평가

#alpha값 최적화 후 학습/예측/평가 수행, 시각화

```
# 앞의 최적화 alpha값으로 학습데이터로 학습, 테스트 데이터로 예측 및 평가 수행.
lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
ridge_reg = Ridge(alpha=12)
ridge_reg.fit(X_train, y_train)
lasso_reg = Lasso(alpha=0.001)
lasso_reg.fit(X_train, y_train)

# 모든 모델의 RMSE 출력
models = [lr_reg, ridge_reg, lasso_reg]
get_rmse(models)

# 모든 모델의 회귀 계수 시각화
models = [lr_reg, ridge_reg, lasso_reg]
visualize_coefficient(models)
```

3] ✓ 0.9s

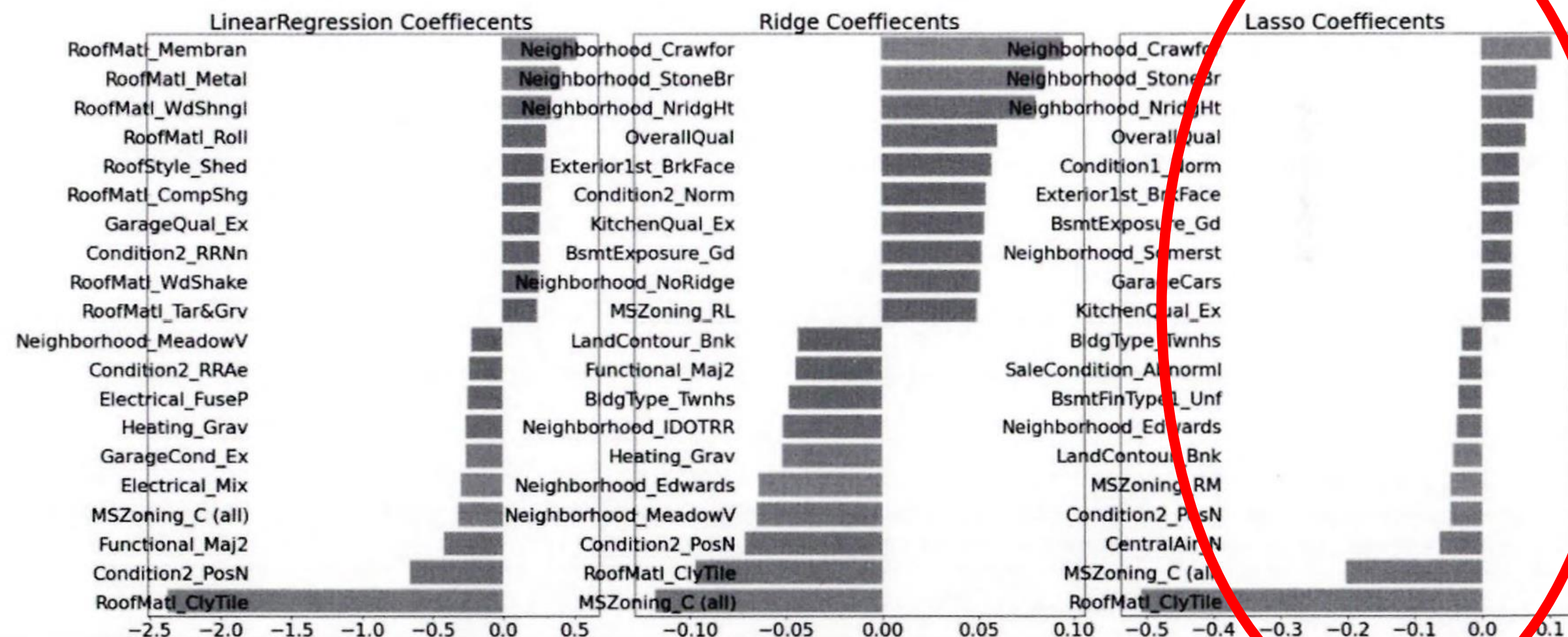
LinearRegression 로그 변환된 RMSE: 0.132

Ridge 로그 변환된 RMSE: 0.124

Lasso 로그 변환된 RMSE: 0.12

예측 성능 향상

형태는 비슷,
회귀 계수 값 작음



선형 회귀 모델 학습/예측/평가

#데이터 세트 추가 가공 - ① 피쳐 데이터 세트의 왜곡 조정

```
from scipy.stats import skew

# object가 아닌 숫자형 피쳐의 칼럼 index 객체 추출.
features_index = house_df.dtypes[house_df.dtypes != 'object'].index
# house_df에 칼럼 index를 [ ]로 입력하면 해당하는 칼럼 데이터 세트 반환. apply lambda로 skew( ) 호출
skew_features = house_df[features_index].apply(lambda x : skew(x))
# skew(왜곡) 정도가 1 이상인 칼럼만 추출.
skew_features_top = skew_features[skew_features > 1]
print(skew_features_top.sort_values(ascending=False))
```

✓ 0.0s

MiscVal	24.451640
PoolArea	14.813135
LotArea	12.195142
3SsnPorch	10.293752
LowQualFinSF	9.002080
KitchenAbvGr	4.483784
BsmtFinSF2	4.250888
ScreenPorch	4.117977
BsmtHalfBath	4.099186
EnclosedPorch	3.086696
MasVnrArea	2.673661
LotFrontage	2.382499
OpenPorchSF	2.361912
BsmtFinSF1	1.683771
WoodDeckSF	1.539792
TotalBsmtSF	1.522688
MSSubClass	1.406210
1stFlrSF	1.375342
GrLivArea	1.365156
dtype: float64	

- skew() 함수를 이용해 왜곡된 정도 추출
- 1 이상의 값 반환하는 피쳐만 추출해 로그 변환
- 주의) 원-핫 인코딩된 카테고리 숫자형 피쳐 제외

```
house_df[skew_features_top.index] = np.log1p(house_df[skew_features_top.index])
```

✓ 0.0s

회귀 트리 모델 학습/예측/평가

```
# 왜곡 정도가 높은 피쳐들을 로그 변환 했으므로 다시 원-핫 인코딩 적용 및 피쳐/타겟 데이터 셋 생성
house_df_ohe = pd.get_dummies(house_df)
y_target = house_df_ohe['SalePrice']
X_features = house_df_ohe.drop('SalePrice',axis=1, inplace=False)
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.2, random_state=156)

# 피쳐들을 로그 변환 후 다시 최적 하이퍼 파라미터와 RMSE 출력
ridge_params = { 'alpha':[0.05, 0.1, 1, 5, 8, 10, 12, 15, 20] }
lasso_params = { 'alpha':[0.001, 0.005, 0.008, 0.05, 0.03, 0.1, 0.5, 1,5, 10] }
best_ridge = print_best_params(ridge_reg, ridge_params)
best_lasso = print_best_params(lasso_reg, lasso_params)
```

✓ 1.7s

학습, 테스트 데이터로 예측 및 평가 수행.

0.1418 ->
Ridge 5 CV 시 최적 평균 RMSE 값 0.1275 최적 alpha:{'alpha': 10}
Lasso 5 CV 시 최적 평균 RMSE 값 0.1252 최적 alpha:{'alpha': 0.001}

0.142 ->

```
lr_reg = LinearRegression(),
lr_reg.fit(X_train, y_train)
```

```
# 모든 모델의 RMSE 출력
models = [lr_reg, ridge_reg, lasso_reg]
get_rmses(models)

# 모든 모델의 회귀 계수 시각화
models = [lr_reg, ridge_reg, lasso_reg]
visualize_coefficient(models)
```

✓ 0.9s

회귀 트리 모델 학습/예측/평가

```
# 앞의 최적화 alpha값으로 학습데이터로 학습, 테스트 데이터로 예측 및 평가 수행.
```

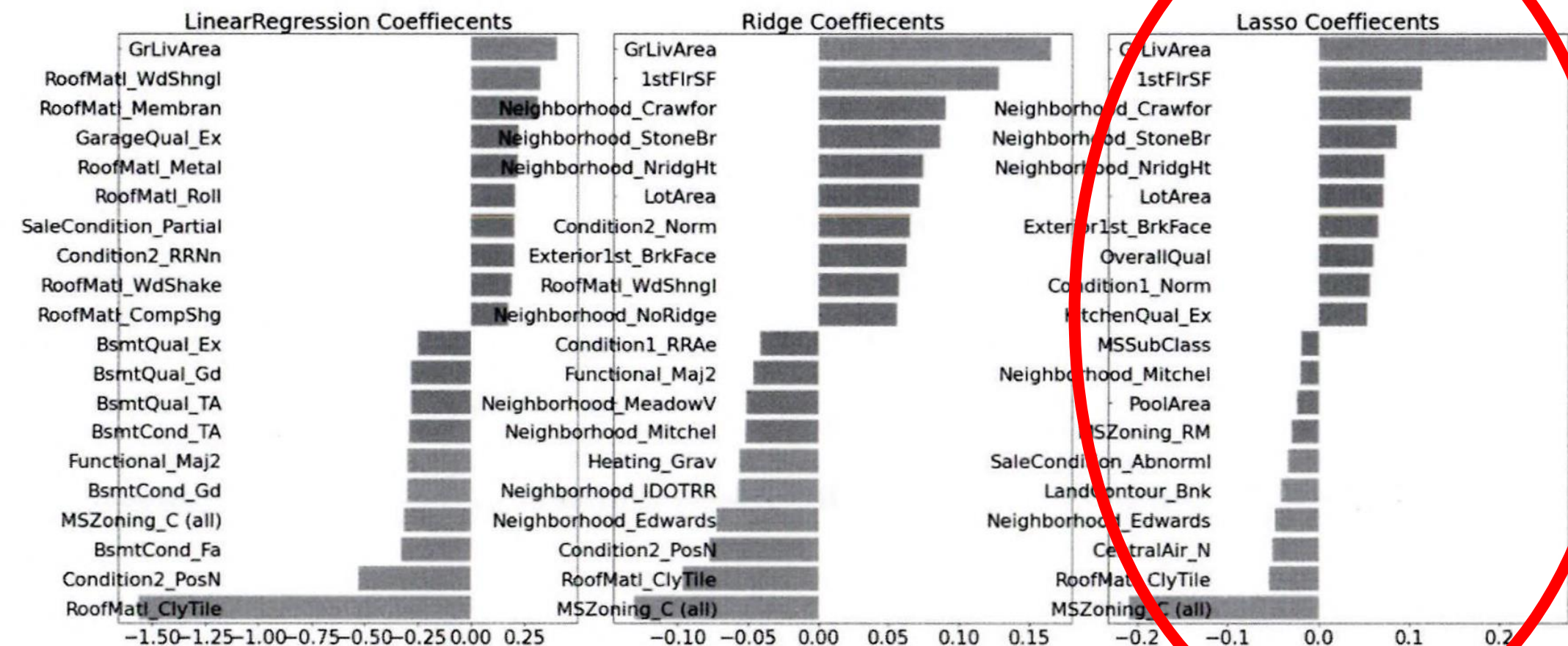
```
lr_reg = LinearRegression()  
lr_reg.fit(X_train, y_train)  
ridge_reg = Ridge(alpha=10)  
ridge_reg.fit(X_train, y_train)  
lasso_reg = Lasso(alpha=0.001)  
lasso_reg.fit(X_train, y_train)
```

```
# 모든 모델의 RMSE 출력
```

```
models = [lr_reg, ridge_reg, lasso_reg]  
get_rmse(models)
```

```
# 모든 모델의 회귀 계수 시각화
```

```
models = [lr_reg, ridge_reg, lasso_reg]  
visualize_coefficient(models)
```

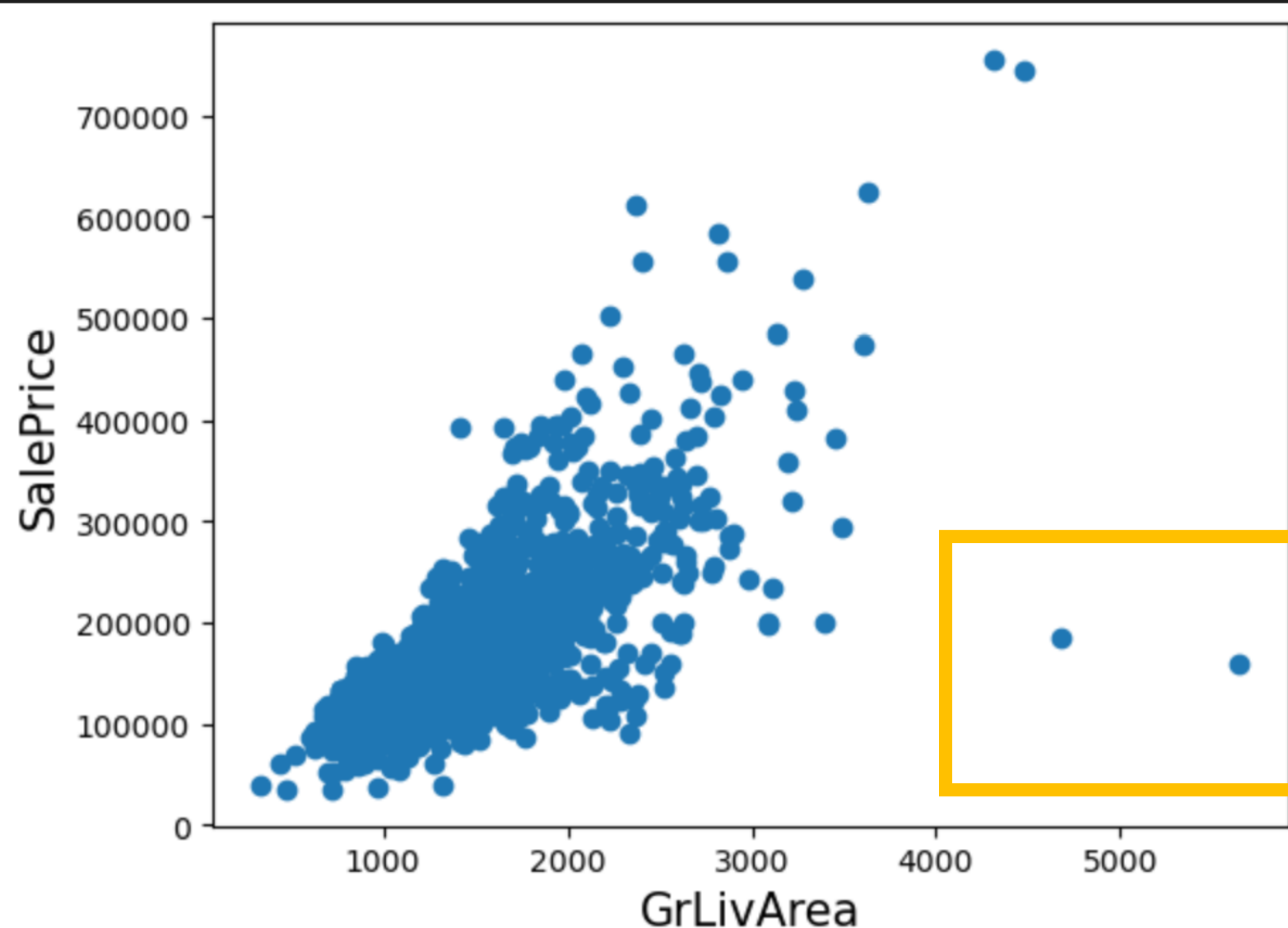


선형 회귀 모델 학습/예측/평가

#데이터 세트 추가 가공 - ② 이상치 데이터 처리

```
plt.scatter(x = house_df_org['GrLivArea'], y = house_df_org['SalePrice'])  
plt.ylabel('SalePrice', fontsize=15)  
plt.xlabel('GrLivArea', fontsize=15)  
plt.show()
```

✓ 0.2s



```
# GrLivArea와 SalePrice 모두 로그 변환되었으므로 이를 반영한 조건 생성.  
cond1 = house_df_ohe['GrLivArea'] > np.log1p(4000)  
cond2 = house_df_ohe['SalePrice'] < np.log1p(500000)  
outlier_index = house_df_ohe[cond1 & cond2].index
```

```
print('아웃라이어 레코드 index:', outlier_index.values)  
print('아웃라이어 삭제 전 house_df_ohe shape:', house_df_ohe.shape)
```

```
# DataFrame의 index를 이용하여 아웃라이어 레코드 삭제.  
house_df_ohe.drop(outlier_index, axis=0, inplace=True)  
print('아웃라이어 삭제 후 house_df_ohe shape:', house_df_ohe.shape)
```

✓ 0.0s

이상치 레코드 index : [523 1298]

이상치 삭제 전 house_df_ohe shape: (1460, 271)

이상치 삭제 후 house_df_ohe shape: (1458, 271)

→ 이상치

선형 회귀 모델 학습/예측/평가

#데이터 세트 추가 가공 - ② 이상치 데이터 처리

```
y_target = house_df_ohe['SalePrice']
X_features = house_df_ohe.drop('SalePrice',axis=1, inplace=False)
X_train, X_test, y_train, y_test = train_test_split(X_features, y_target, test_size=0.2, random_state=156)

ridge_params = { 'alpha':[0.05, 0.1, 1, 5, 8, 10, 12, 15, 20] }
lasso_params = { 'alpha':[0.001, 0.005, 0.008, 0.05, 0.03, 0.1, 0.5, 1,5, 10] }
best_ridge = print_best_params(ridge_reg, ridge_params)
best_lasso = print_best_params(lasso_reg, lasso_params)
```

✓ 1.9s

Ridge 5 CV 시 최적 평균 RMSE 값: 0.1125, 최적 alpha:{'alpha': 8}

Lasso 5 CV 시 최적 평균 RMSE 값: 0.1122, 최적 alpha:{'alpha': 0.001}

앞의 최적화 alpha값으로 학습데이터로 학습, 테스트 데이터로 예측 및 평가 수행.

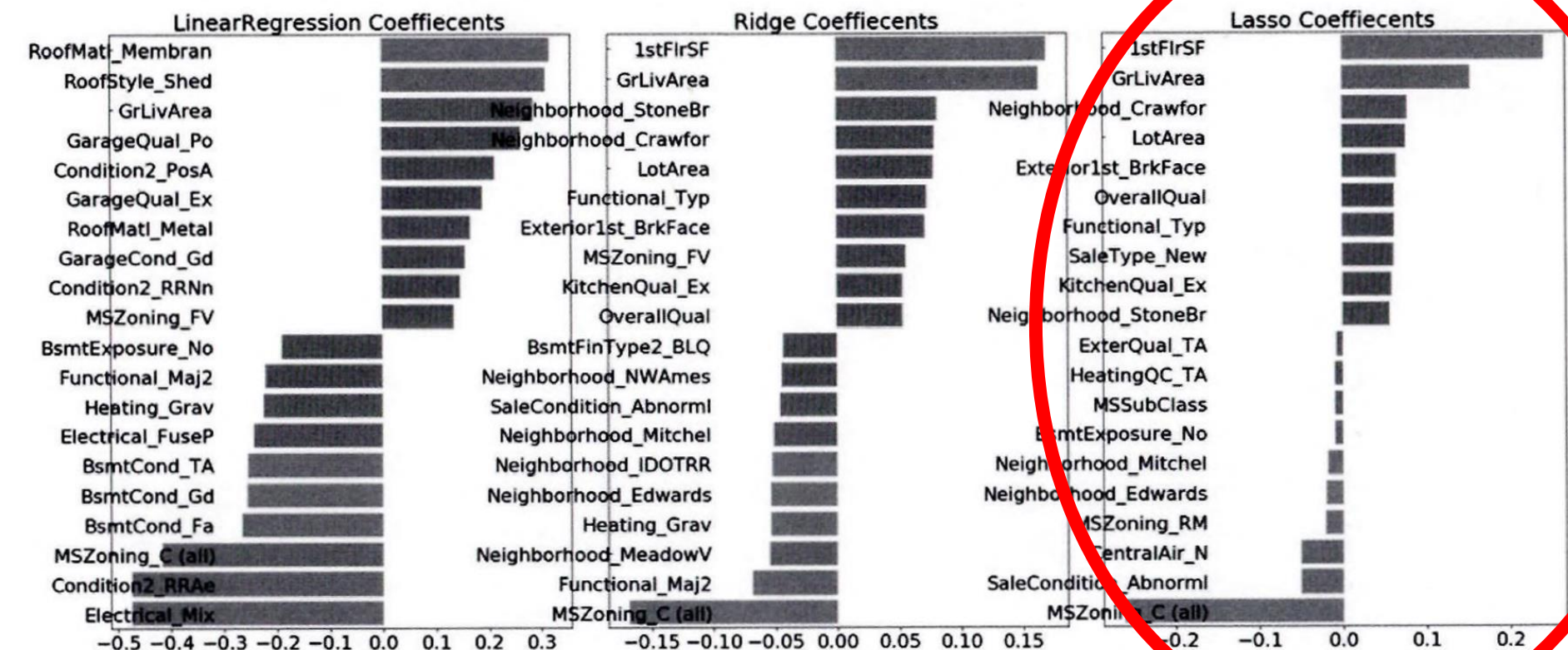
```
lr_reg = LinearRegression()
lr_reg.fit(X_train, y_train)
ridge_reg = Ridge(alpha=8)
ridge_reg.fit(X_train, y_train)
lasso_reg = Lasso(alpha=0.001)
lasso_reg.fit(X_train, y_train)
```

모든 모델의 RMSE 출력

```
models = [lr_reg, ridge_reg, lasso_reg]
get_rmse(models)
```

모든 모델의 회귀 계수 시각화

```
models = [lr_reg, ridge_reg, lasso_reg]
visualize_coefficient(models)
```



회귀 모델의 학습/예측/평가

```
from xgboost import XGBRegressor

xgb_params = {'n_estimators':[1000]}
xgb_reg = XGBRegressor(n_estimators=1000, learning_rate=0.05,
                        colsample_bytree=0.5, subsample=0.8)
best_xgb = print_best_params(xgb_reg, xgb_params)
```

✓ 9.6s

XG Boost 회귀 트리 적용

XGBRegressor 5 CV 시 최적 평균 RMSE 값: 0.1178, 최적 alpha: {'n_estimators': 1000}

```
from lightgbm import LGBMRegressor

lgbm_params = {'n_estimators':[1000]}
lgbm_reg = LGBMRegressor(n_estimators=1000, learning_rate=0.05, num_leaves=4,
                          subsample=0.6, colsample_bytree=0.4, reg_lambda=10, n_jobs=-1)
best_lgbm = print_best_params(lgbm_reg, lgbm_params)
```

✓ 5.4s

Light GBM 회귀 트리 적용

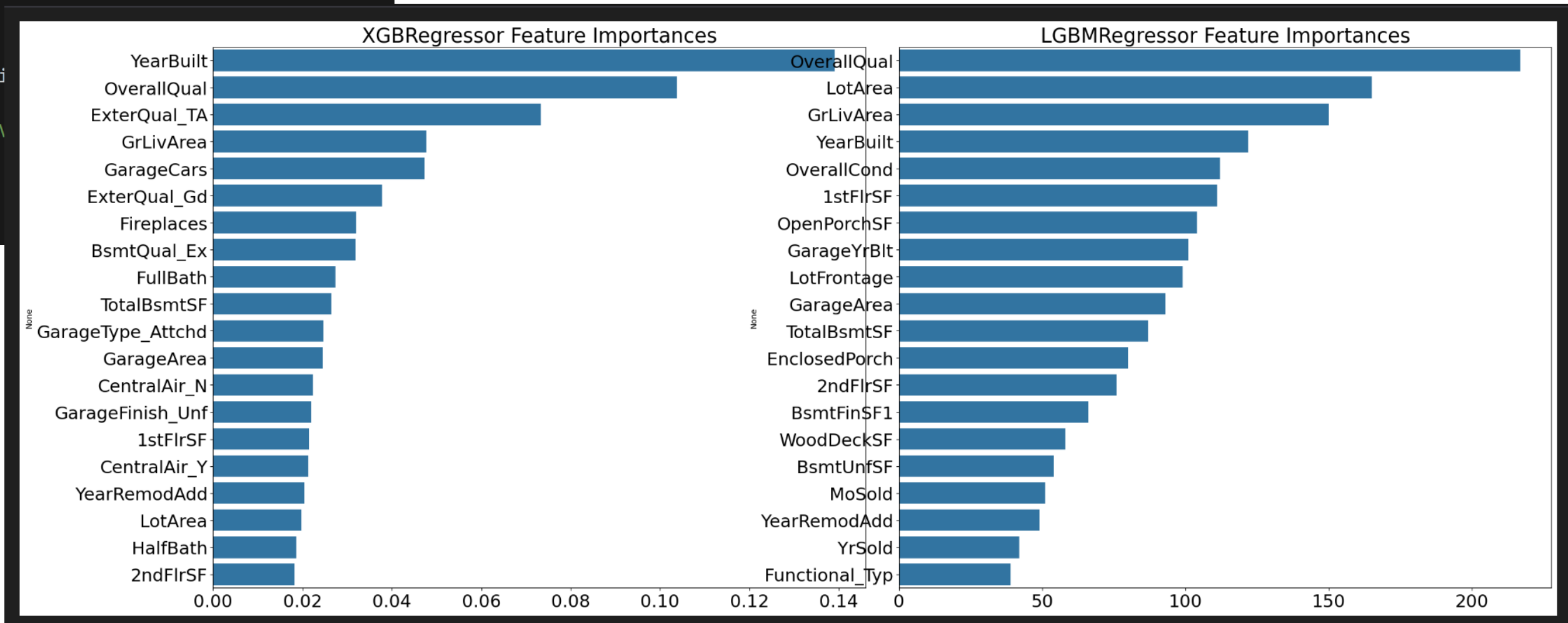
LGBMRegressor 5 CV 시 최적 평균 RMSE 값: 0.1163, 최적 alpha: {'n_estimators': 1000}

회귀 모델의 학습/예측/평가

```
# 모델의 중요도 상위 20개의 피처명과 그때의 중요도값을 Series로 반환
def get_top_features(model):
    ftr_importances_values = model.feature_importances_
    ftr_importances = pd.Series(ftr_importances_values, index=X_features.columns )
    ftr_top20 = ftr_importances.sort_values(ascending=False)[:20]
    return ftr_top20

def visualize_ftr_importances(models):
    # 2개 회귀 모델의 시각화를 위해 2개의 컬럼을 가지는 subplot 생성
    fig, axs = plt.subplots(figsize=(24,10),nrows=1, ncols=2)
    fig.tight_layout()
    # 입력인자로 받은 list객체인 models에서 차례로 model을 추출하여 피처 중요도 시각화.
    for i_num, model in enumerate(models):
        # 중요도 상위 20개의 피처명과 그때의 중요도값 추출
        ftr_top20 = get_top_features(model)
        axs[i_num].set_title(model.__class__.__name__+' Feature Importances', size=25)
        #font 크기 조정.
        for label in (axs[i_num].get_xticklabels() +
                      axs[i_num].get_yticklabels()):
            label.set_fontsize(22)
        sns.barplot(x=ftr_top20.values, y=ftr_top20.index, ax=axs[i_num])

# 앞 예제에서 print_best_params( )가 반환한 GridSearchCV
models = [best_xgb, best_lgbm]
visualize_ftr_importances(models)
plt.show()
```



회귀 모델의 예측 결과 혼합을 통한 최종 예측

#릿지 모델과 라쏘 모델의 혼합

```
def get_rmse_pred(preds):  
    for key in preds.keys():  
        pred_value = preds[key]  
        mse = mean_squared_error(y_test , pred_value)  
        rmse = np.sqrt(mse)  
        print('{0} 모델의 RMSE: {1}'.format(key, rmse))
```

개별 모델의 학습

```
ridge_reg = Ridge(alpha=8)  
ridge_reg.fit(X_train, y_train)  
lasso_reg = Lasso(alpha=0.001)  
lasso_reg.fit(X_train, y_train)
```

개별 모델 예측

```
ridge_pred = ridge_reg.predict(X_test)  
lasso_pred = lasso_reg.predict(X_test)
```

개별 모델 예측값 혼합으로 최종 예측값 도출

```
pred = 0.4 * ridge_pred + 0.6 * lasso_pred
```

```
preds = {'최종 혼합': pred,  
        'Ridge': ridge_pred,  
        'Lasso': lasso_pred}
```

#최종 혼합 모델, 개별모델의 RMSE 값 출력

```
get_rmse_pred(preds)
```

✓ 0.0s

최종 혼합 모델의 RMSE: 0.10007930884470506

Ridge 모델의 RMSE: 0.10345177546603253

Lasso 모델의 RMSE: 0.10024170460890032

회귀 모델의 예측 결과 혼합을 통한 최종 예측

#XG Boost와 LightBGM 혼합

```
xgb_reg = XGBRegressor(n_estimators=1000, learning_rate=0.05,  
                        colsample_bytree=0.5, subsample=0.8)  
lgbm_reg = LGBMRegressor(n_estimators=1000, learning_rate=0.05, num_leaves=4,  
                          subsample=0.6, colsample_bytree=0.4, reg_lambda=10, n_jobs=-1)  
xgb_reg.fit(X_train, y_train)  
lgbm_reg.fit(X_train, y_train)  
xgb_pred = xgb_reg.predict(X_test)  
lgbm_pred = lgbm_reg.predict(X_test)  
  
pred = 0.5 * xgb_pred + 0.5 * lgbm_pred  
preds = {'최종 혼합': pred,  
         'XGBM': xgb_pred,  
         'LGBM': lgbm_pred}  
  
get_rmse_pred(preds)  
✓ 1.9s
```

최종 혼합 모델의 RMSE: 0.10170077353447762

XGBM 모델의 RMSE: 0.10738295638346222

LGBM 모델의 RMSE: 0.10382510019327311

스택킹 앙상블 모델을 통한 회귀 예측

```
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error

# 개별 기반 모델에서 최종 메타 모델이 사용할 학습 및 테스트용 데이터를 생성하기 위한 함수.
def get_stacking_base_datasets(model, X_train_n, y_train_n, X_test_n, n_folds ):
    # 지정된 n_folds값으로 KFold 생성.
    kf = KFold(n_splits=n_folds, shuffle=False)
    #추후에 메타 모델이 사용할 학습 데이터 반환을 위한 넘파이 배열 초기화
    train_fold_pred = np.zeros((X_train_n.shape[0] ,1 ))
    test_pred = np.zeros((X_test_n.shape[0],n_folds))
    print(model.__class__.__name__ , ' model 시작 ')

    for folder_counter , (train_index, valid_index) in enumerate(kf.split(X_train_n)):
        #입력된 학습 데이터에서 기반 모델이 학습/예측할 폴드 데이터 셋 추출
        print('\t 폴드 세트: ',folder_counter,' 시작 ')
        X_tr = X_train_n[train_index]
        y_tr = y_train_n[train_index]
        X_te = X_train_n[valid_index]

        #폴드 세트 내부에서 다시 만들어진 학습 데이터로 기반 모델의 학습 수행.
        model.fit(X_tr , y_tr)
        #폴드 세트 내부에서 다시 만들어진 검증 데이터로 기반 모델 예측 후 데이터 저장.
        train_fold_pred[valid_index, :] = model.predict(X_te).reshape(-1,1)
        #입력된 원본 테스트 데이터를 폴드 세트내 학습된 기반 모델에서 예측 후 데이터 저장.
        test_pred[:, folder_counter] = model.predict(X_test_n)

    # 폴드 세트 내에서 원본 테스트 데이터를 예측한 데이터를 평균하여 테스트 데이터로 생성
    test_pred_mean = np.mean(test_pred, axis=1).reshape(-1,1)

    #train_fold_pred는 최종 메타 모델이 사용하는 학습 데이터, test_pred_mean은 테스트 데이터
    return train_fold_pred , test_pred_mean
```

✓ 0.0s

스택킹 앙상블 모델을 통한 회귀 예측

#get_stacking_base_datasets()를 모델별로 적용해 학습 데이터 세트와 테스트 피쳐 데이터 세트 추출

```
# get_stacking_base_datasets( )은 넘파이 ndarray를 인자로 사용하므로 DataFrame을 넘파이로 변환.
X_train_n = X_train.values
X_test_n = X_test.values
y_train_n = y_train.values

# 각 개별 기반(Base)모델이 생성한 학습용/테스트용 데이터 반환.
ridge_train, ridge_test = get_stacking_base_datasets(ridge_reg, X_train_n, y_train_n, X_test_n, 5)
lasso_train, lasso_test = get_stacking_base_datasets(lasso_reg, X_train_n, y_train_n, X_test_n, 5)
xgb_train, xgb_test = get_stacking_base_datasets(xgb_reg, X_train_n, y_train_n, X_test_n, 5)
lgbm_train, lgbm_test = get_stacking_base_datasets(lgbm_reg, X_train_n, y_train_n, X_test_n, 5)
```

✓ 4.3s

#학습 데이터 세트와 테스트 피쳐 데이터 세트를 결합해 최종 메타 모델에 적용

```
# 개별 모델이 반환한 학습 및 테스트용 데이터 세트를 Stacking 형태로 결합.
Stack_final_X_train = np.concatenate((ridge_train, lasso_train,
                                       xgb_train, lgbm_train), axis=1)
Stack_final_X_test = np.concatenate((ridge_test, lasso_test,
                                       xgb_test, lgbm_test), axis=1)

# 최종 메타 모델은 라쏘 모델을 적용.
meta_model_lasso = Lasso(alpha=0.0005)

#기반 모델의 예측값을 기반으로 새롭게 만들어진 학습 및 테스트용 데이터로 예측하고 RMSE 측정.
meta_model_lasso.fit(Stack_final_X_train, y_train)
final = meta_model_lasso.predict(Stack_final_X_test)
mse = mean_squared_error(y_test , final)
rmse = np.sqrt(mse)
print('스태킹 회귀 모델의 최종 RMSE 값은:', rmse)
```

✓ 0.0s

스태킹 회귀 모델의 최종 RMSE 값은: 0.09799152965189684

Avocado Price Regression w/ PyCaret & EDA



#00 노트북 소개

❖ 🥑 아보카도 가격 예측

- ❑ 목표
 - 아보카도 가격을 다양한 변수(지역, 종류, 연도, 판매량 등)로 예측
- ❑ 방법:
 - 회귀(Regression)기법
- ❑ AutoML Library's PyCaret 활용
- ❑ 주요 작업:
 1. EDA 수행
 2. 데이터 전처리 (PyCaret 적용전)
 3. PyCaret 회귀 모델 적용

#01 데이터세트 소개

변수 이름	설명	예시 값
Index	데이터 인덱스	1, 2, 3, ...
Date	관측 날짜 (yyyy-mm-dd 포맷)	2015-12-27, 2015-12-20
AveragePrice	아보카도의 평균 가격	1.33, 0.93
Total Volume	총 판매량 (전체 아보카도 수량)	64,236.62, 118,220.22
4046 , 4225 , 4770	PLU 코드별 아보카도 판매량 (유형별 소분류)	2,695 / 80,596 / 43
Total Bags	전체 가방 판매량 (Small + Large + XLarge Bags)	8,696.87, 9,505.56
Small Bags	소형 가방 판매량	8,603.62
Large Bags	대형 가방 판매량	93.25
XLarge Bags	특대형 가방 판매량	0.00, 33.33
type	아보카도 유형 (일반 or 유기농)	conventional, organic
year	관측 연도	2015, 2016, 2017
region	판매 지역 또는 도시	Albany, Boston, etc.

❖ Dataset Description

- ❑ 총 14개의 변수
- ❑ 3개 범주형 변수(type, year, region)
- ❑ 9개 연속형 변수(AveragePrice, Total Volume, 4046, 4225, 4770, Total Bags, Small Bags, Large Bags, Xlarge Bags)
- ❑ 1개 날짜 변수(Date)
- ❑ 1개 인덱스 변수(Index)

#03 AutoML

❖ AutoML이란?

- ❑ Automated Machine Learning -> 머신러닝의 전체 과정을 자동화
 - 데이터 전처리
 - 모듈 선택
 - 하이퍼파라미터 튜닝
 - 평가
 - 배포

❖ 장점

- ❑ 비전문가: 빠르게 모델을 만들 수 있게 도와줌
- ❑ 전문가: 반복적인 실험 속도 향상에 효과적

❖ 인기 AutoML tools:

- ❑ Google AutoML
- ❑ H2O AutoML
- ❑ PyCaret
- ❑ Auto-sklearn

#04-1 PyCaret 소개

❖ PyCaret이란?

- ❑ 파이썬 기반 AutoML 라이브러리
- ❑ 오픈소스, 로우코드 기반 머신러닝 도구

❖ 지원 작업

- ❑ 분류(Classification)
- ❑ 회귀(Regression)
- ❑ 클러스터링(Clustering)

❖ PyCaret을 왜 사용하는가?

- ❑ 로우코드 : 최소한의 코드로 저체 파이프라인 구성 가능
- ❑ 빠른 실험 : 다양한 모델을 한번에 비교 가능
- ❑ 쉬운 해석 : 내장된 시각화 및 성능 지표 제공
- ❑ 자동 전처리 : 인코딩, 스케일링, 결측치 처리까지 자동 수행
- ❑ 전체 프로세스 지원 : 모델 저장, 예측, 배포까지 한번에 가능

#04-2 PyCaret

❖ PyCaret 설치

```
# Install Libraries
! pip install pycaret
! pip install markupsafe==2.1.3
! pip install jinja2
```

❖ PyCaret의 회귀 모듈 개요

- ❑ `setup()` : 데이터 전처리 설정(타겟 지정, 스케일링, 인코딩 등)
- ❑ `compare_models()` : 여러 회귀 모델 비교 후 최적 모델 반환
- ❑ `evaluate_model()` : 모델 평가
- ❑ `predict_model()` : 예측값 출력 및 성능 평가

```
from pycaret.regression import *

# Step 1: Setup
reg = setup(data=df, target='price')

# Step 2: Compare Models
best_model = compare_models()

# Step 3: Evaluate
evaluate_model(best_model)

# Step 4: Predict
predictions = predict_model(best_model)
```

#05 Avocado Dataset Summary

.. Imported Dataset Info ..

Total Rows: 18249

Total Columns: 14

.. Dataset Details ..

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 18249 entries, 0 to 18248

Data columns (total 14 columns):

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	18249 non-null	int64
1	Date	18249 non-null	object
2	AveragePrice	18249 non-null	float64
3	Total Volume	18249 non-null	float64
4	4046	18249 non-null	float64
5	4225	18249 non-null	float64
6	4770	18249 non-null	float64
7	Total Bags	18249 non-null	float64
8	Small Bags	18249 non-null	float64
9	Large Bags	18249 non-null	float64
10	XLarge Bags	18249 non-null	float64
11	type	18249 non-null	object
12	year	18249 non-null	int64
13	region	18249 non-null	object

dtypes: float64(9), int64(2), object(3)

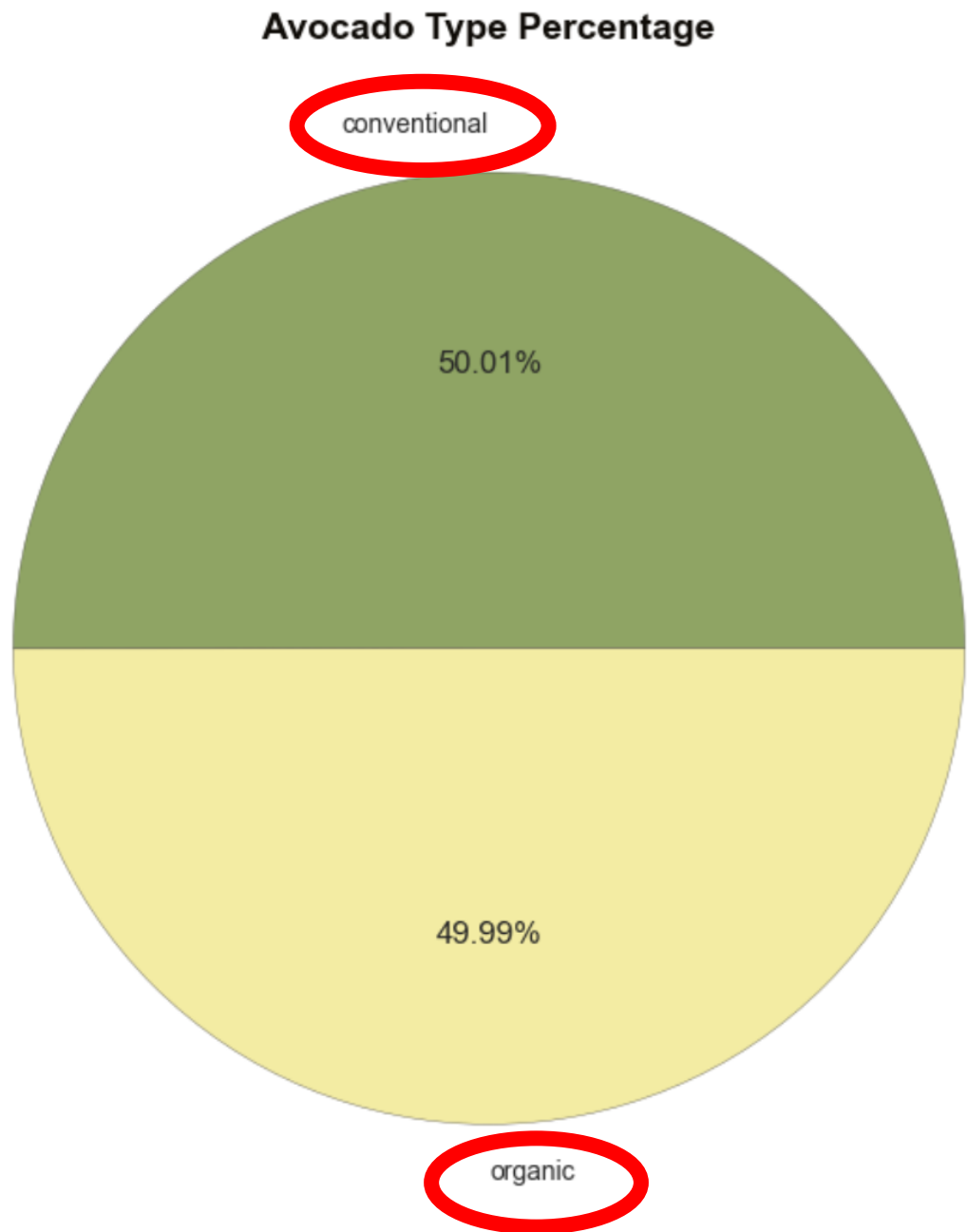
❖ 데이터셋 요약

- ❑ 총 18,249개 행, 14개 열
- ❑ 숫자형 변수 : 11개
- ❑ 범주형 변수: 3개
- ❑ Unnamed: 0는 인덱스 열로 분석에는 불필요

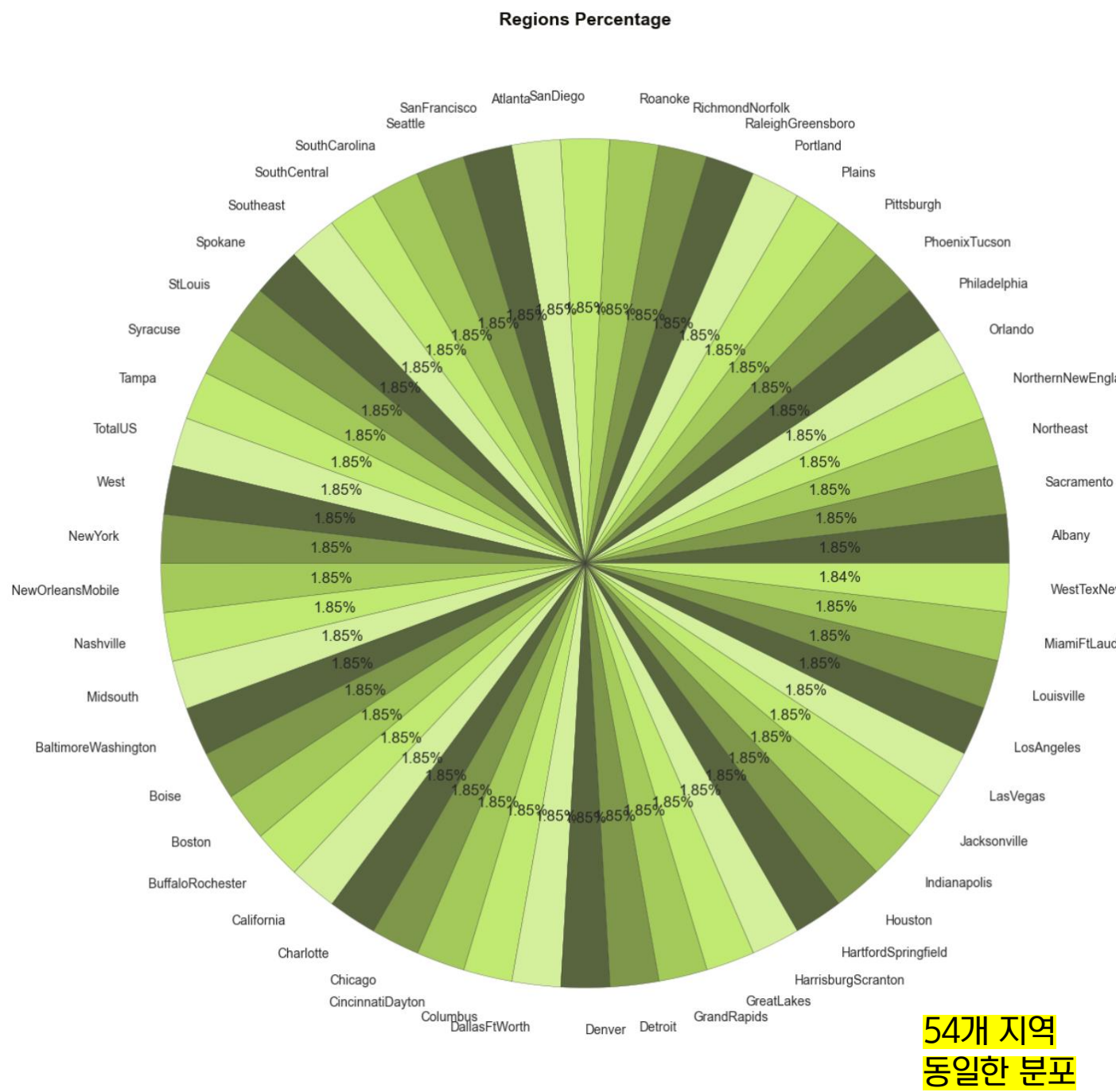
#06-1 시각화 (범주형 변수)

❖ 범주형 변수 시각화

□ 아보카도 유형 분포 분석



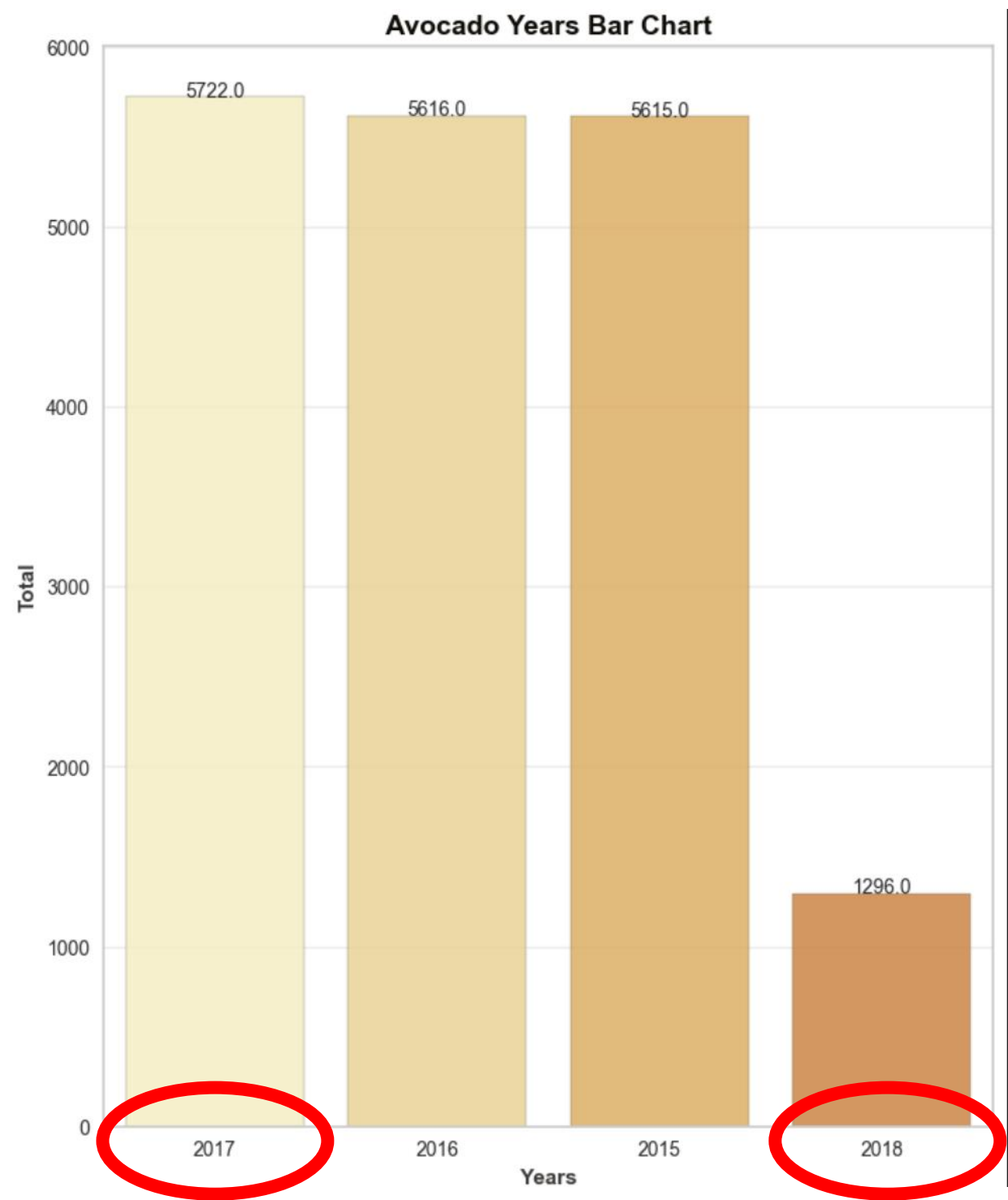
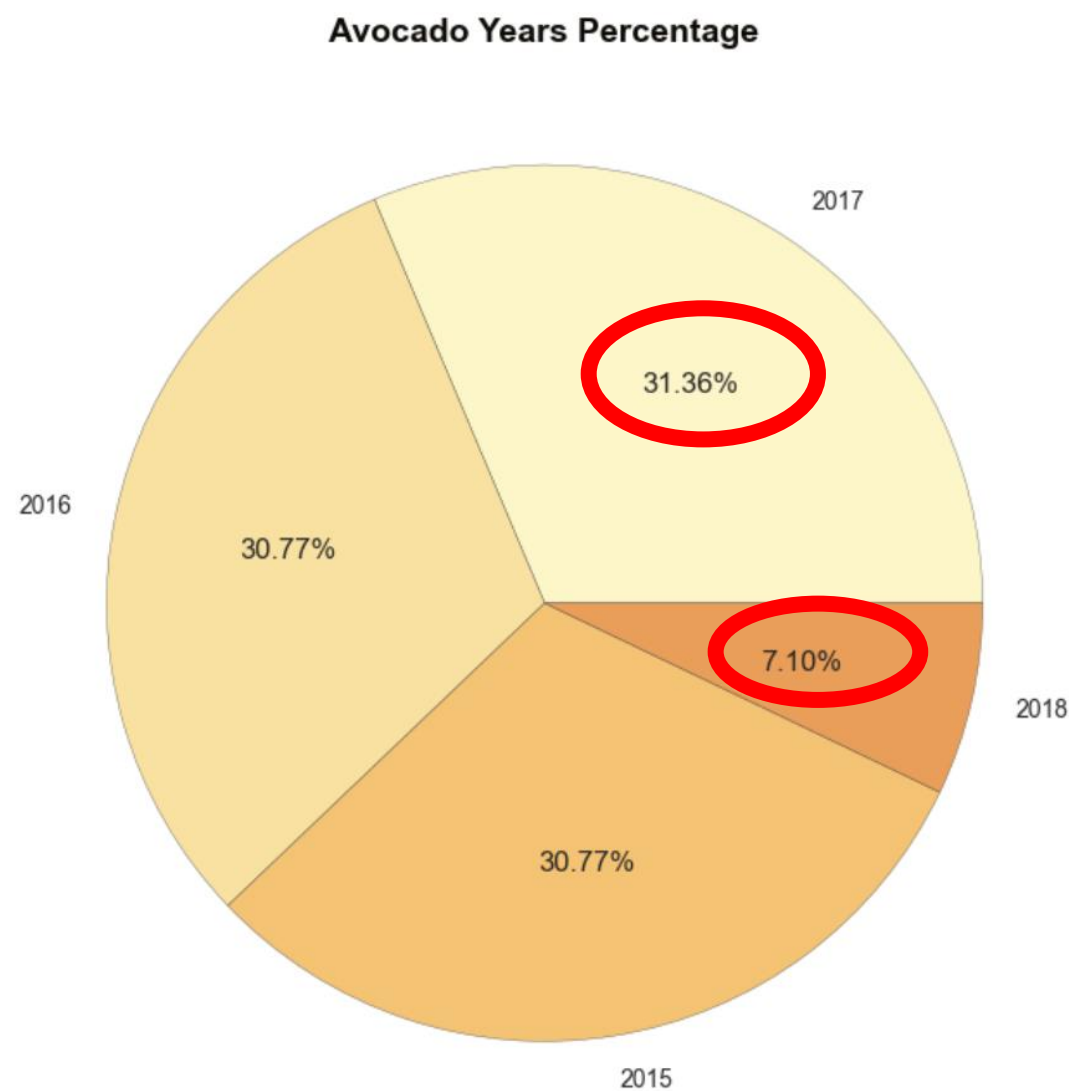
□ 지역별 아보카도 판매 분포 분석



#06-1 시각화 (범주형 변수)

❖ 범주형 변수 시각화

□ 연도별 아보카도 분포 분석



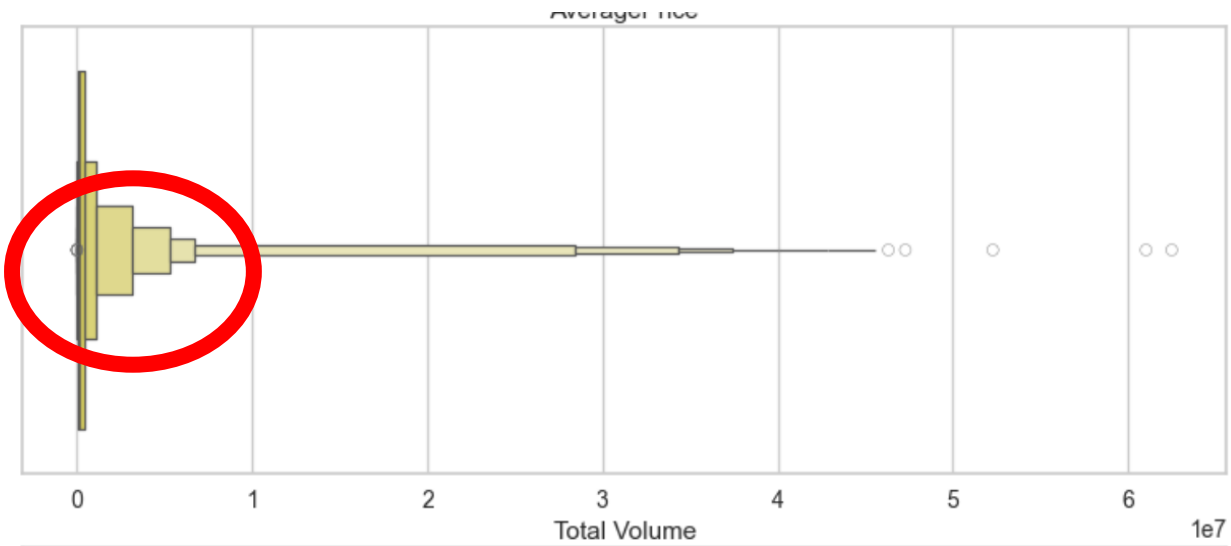
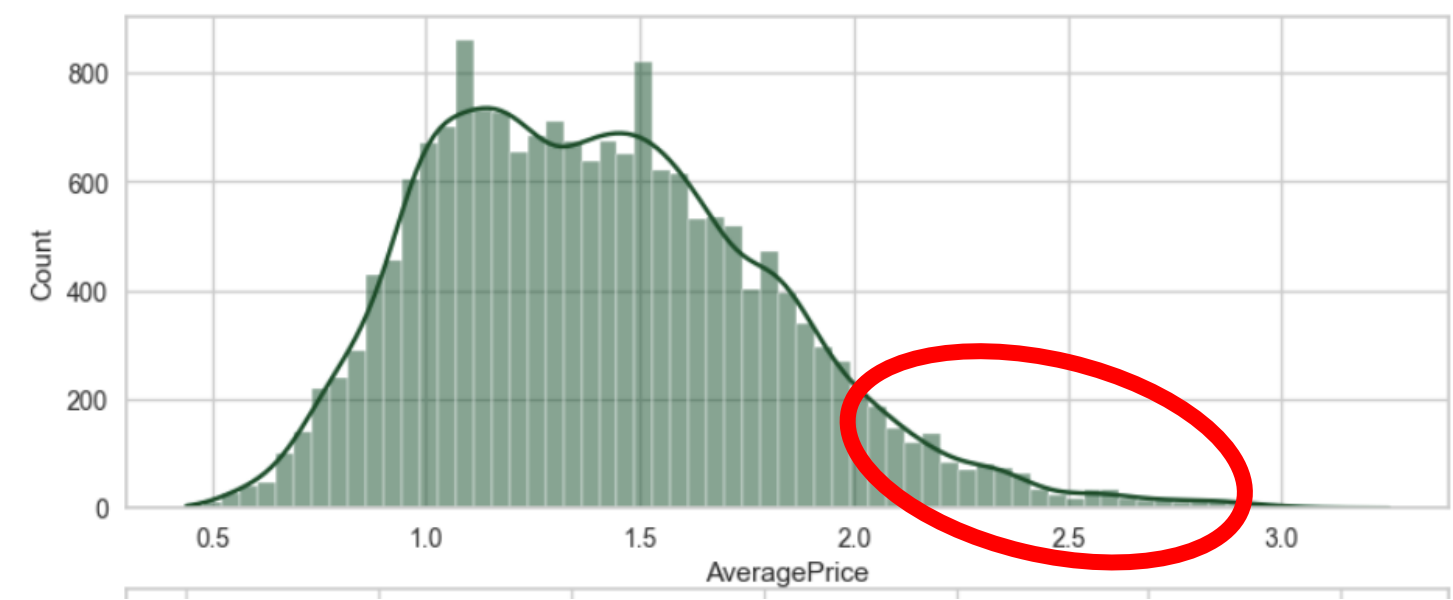
#06-2 시각화 (수치형 변수)

❖ 수치형 변수 시각화

기술통계 요약:

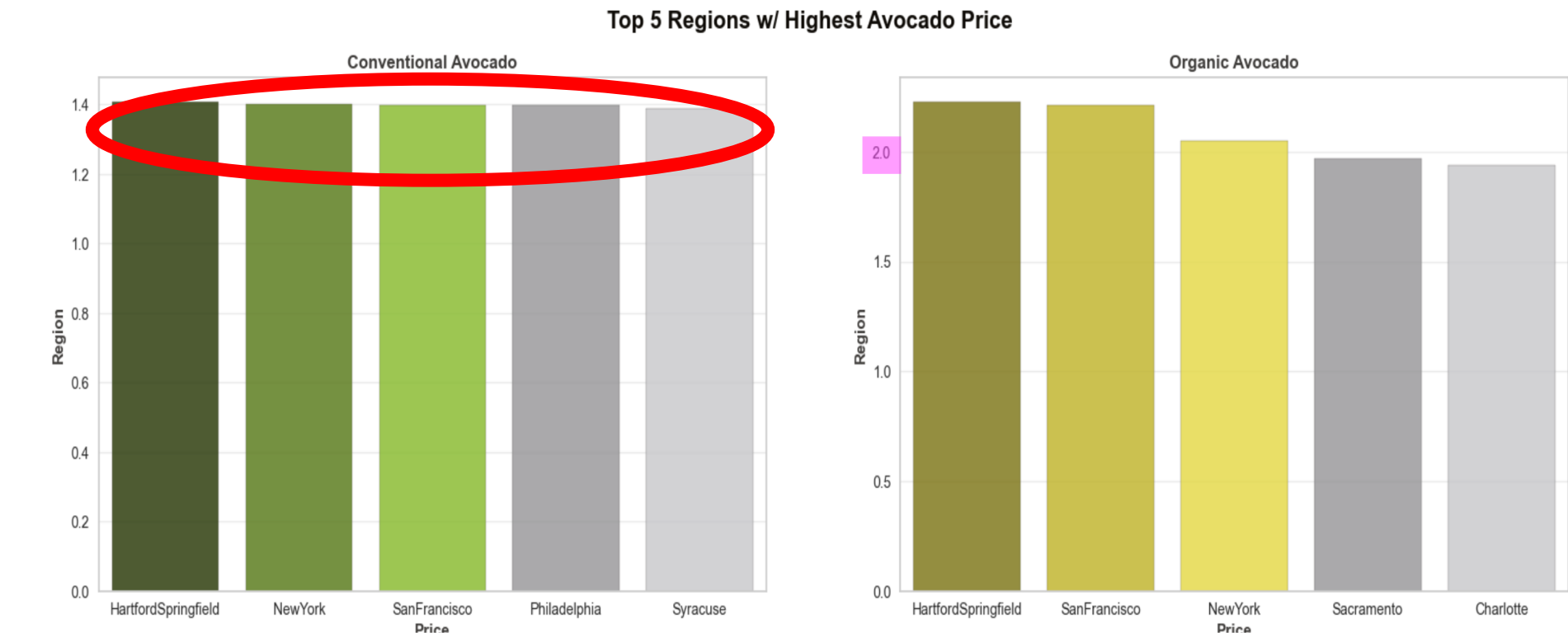
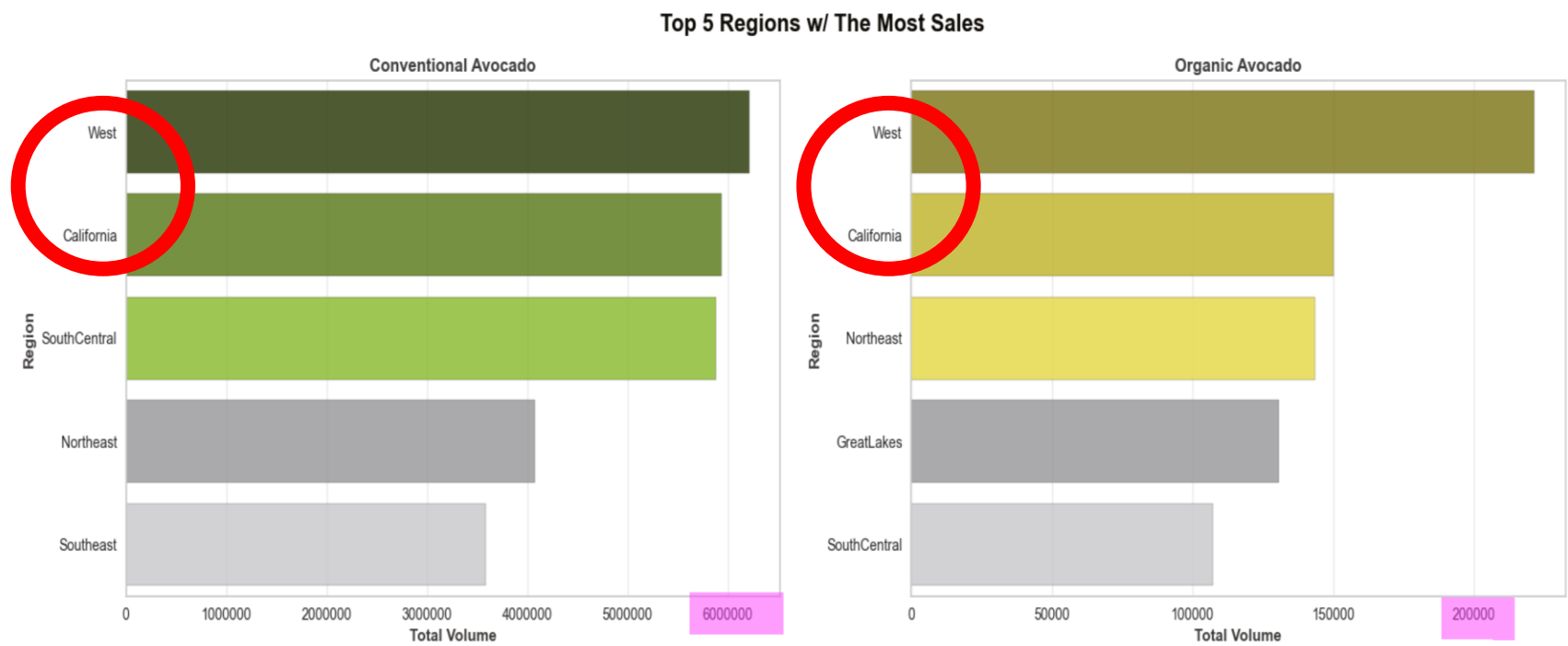
	count	mean	std	min	25%	50%	75%	max
AveragePrice	18249.000000	1.405978	0.402677	0.440000	1.100000	1.370000	1.660000	3.250000
Total Volume	18249.000000	850644.013009	3453545.355399	84.560000	10838.580000	107376.760000	432962.290000	62505646.520000
4046	18249.000000	293008.424531	1264989.081763	0.000000	854.070000	8645.300000	111020.200000	22743616.170000
4225	18249.000000	295154.568356	1204120.401135	0.000000	3008.780000	29061.020000	150206.860000	20470572.610000
4770	18249.000000	22839.735993	107464.068435	0.000000	0.000000	184.990000	6243.420000	2546439.110000
Total Bags	18249.000000	239639.202060	986242.399216	0.000000	5088.640000	39743.830000	110783.370000	19373134.370000
Small Bags	18249.000000	182194.686696	746178.514962	0.000000	2849.420000	26362.820000	83337.670000	13384586.800000
Large Bags	18249.000000	54338.088145	243965.964547	0.000000	127.470000	2647.710000	22029.250000	5719096.610000
XLarge Bags	18249.000000	3106.426507	17692.894652	0.000000	0.000000	0.000000	132.500000	551693.650000

연속형 변수의 분포 분석:



#07 EDA

상위 5개 지역의 아보카도 판매량 비교



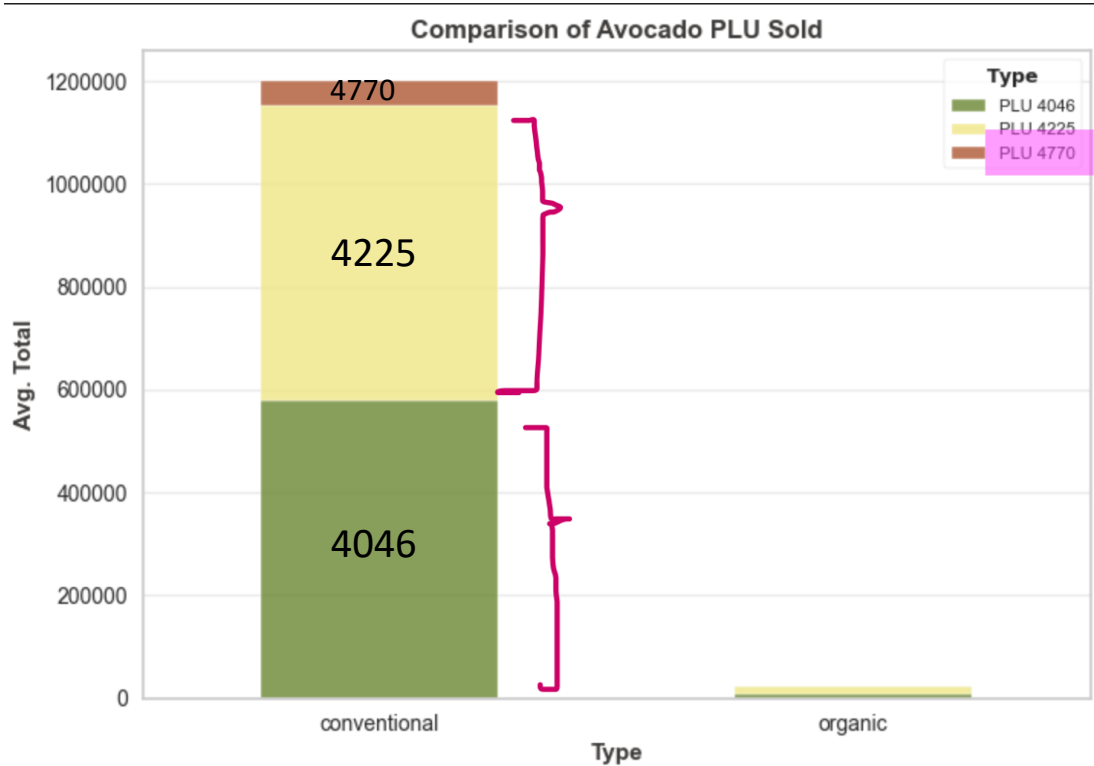
판매량 vs 가방 수 간 산점도



상위 5개 지역의 아보카도 평균 가격 비교

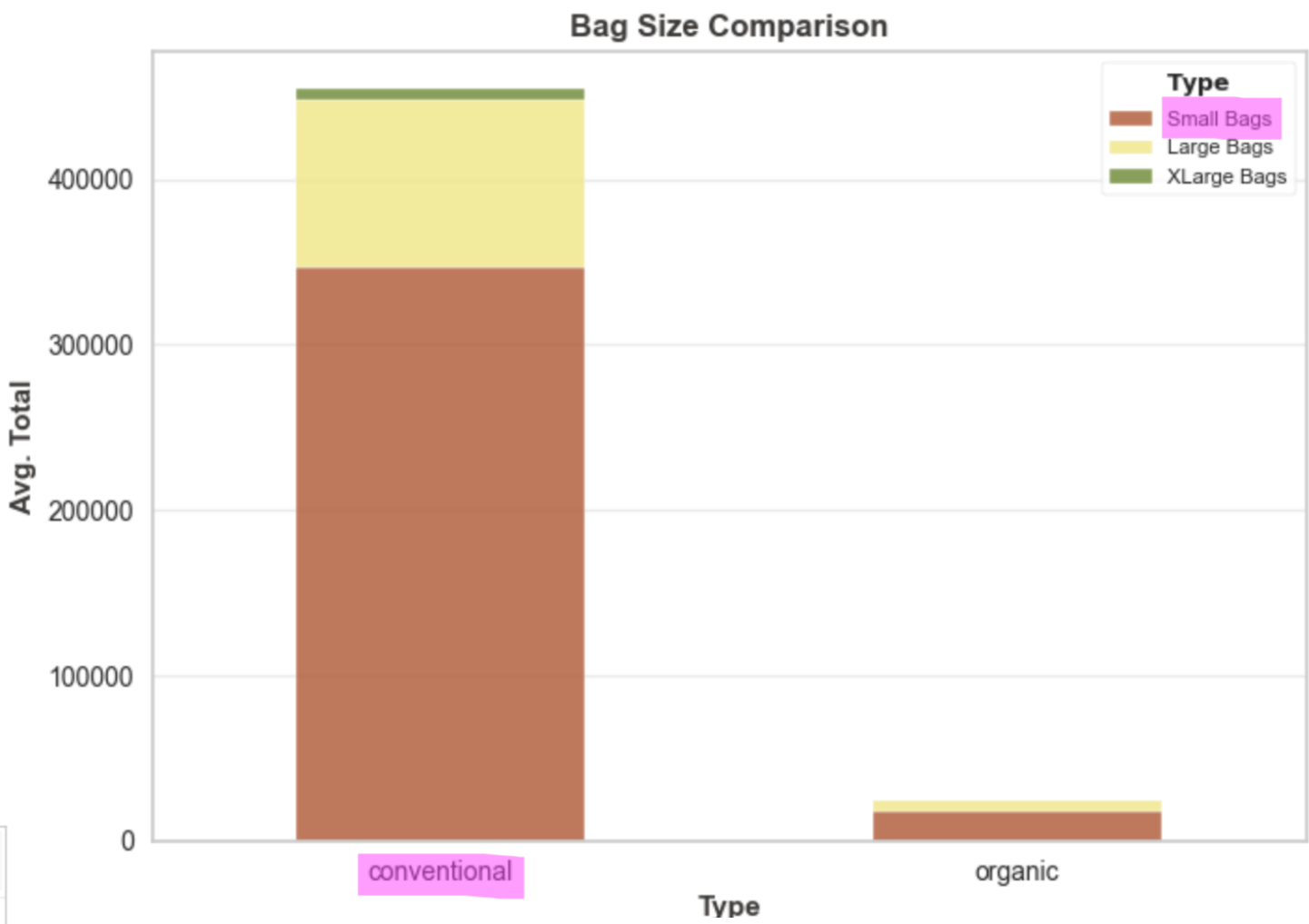
#07 EDA

아보카도 PLU 번호별 판매량 비교

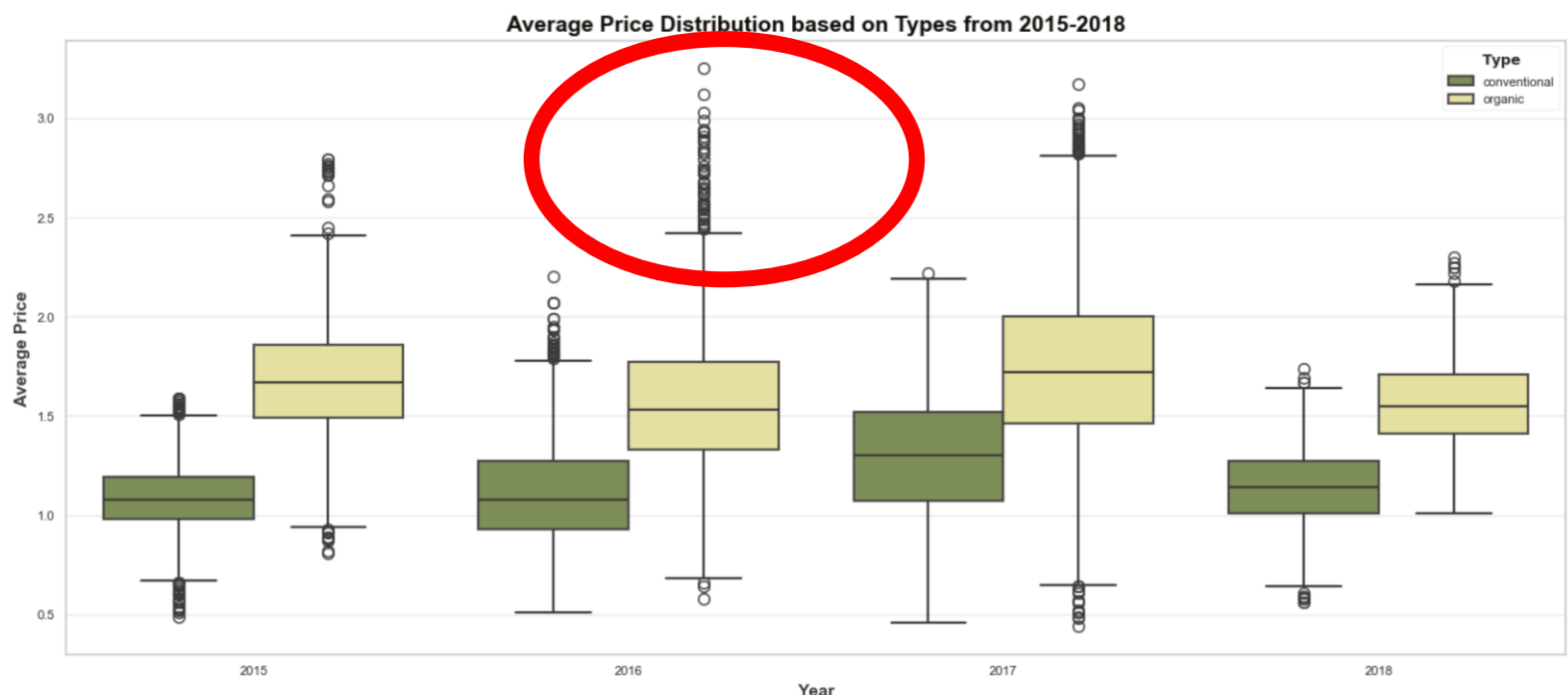


4046: 작은 아보카도
4225: 중간 크기 아보카도
4770: 큰 아보카도

포장 크기별 평균 판매량 비교

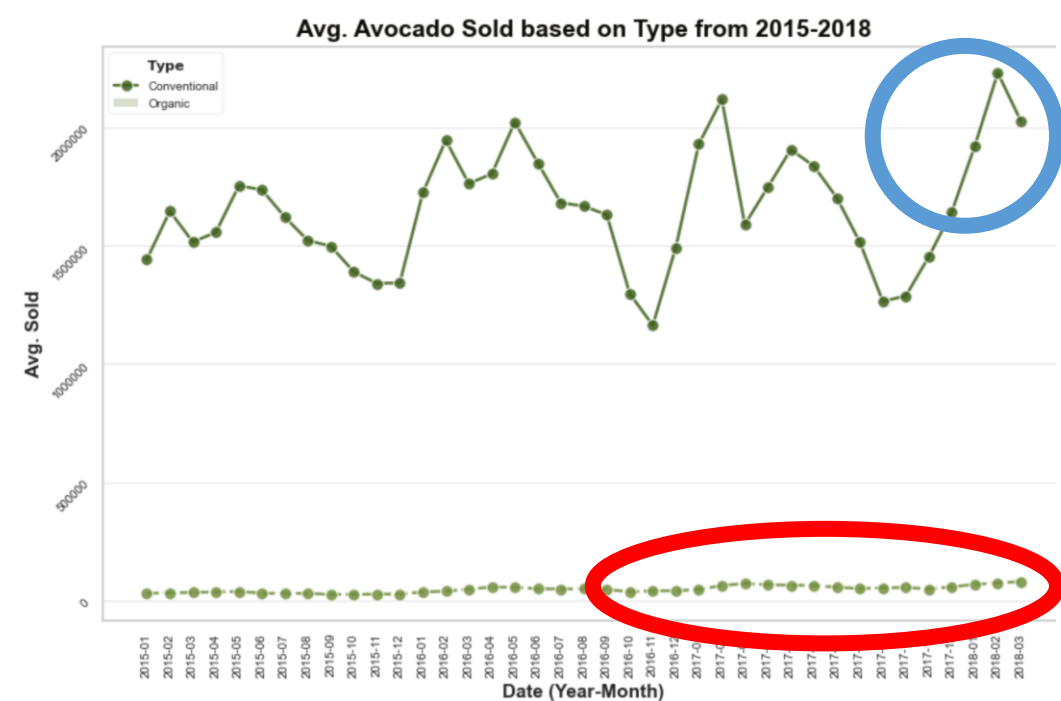


연도별 아보카도 평균 가격 분포(2015~2018)

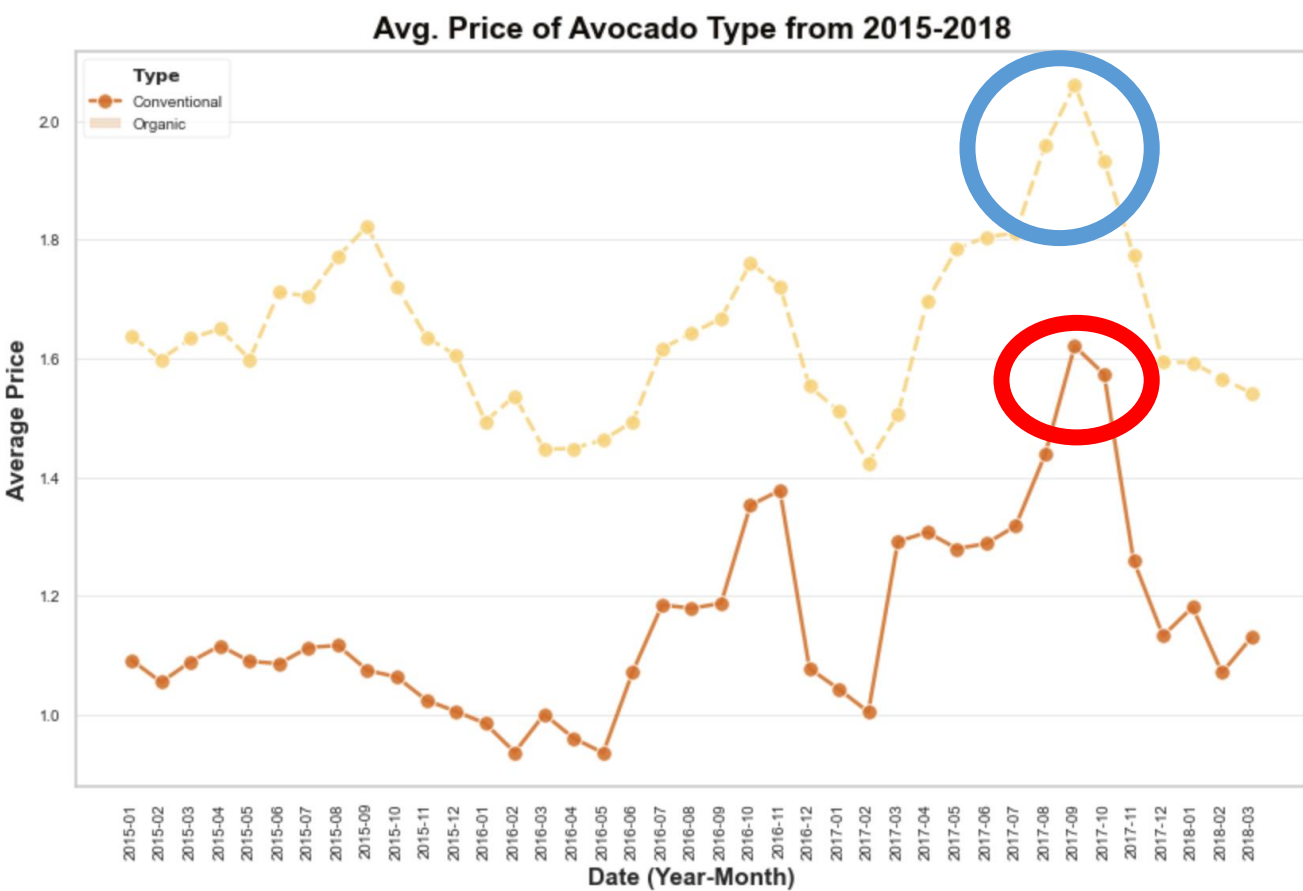


#07 EDA

아보카도 판매량 시계열 분석(2015-2018)



아보카도 평균 가격 시계열 분석(2015-2018)



숫자형 변수 간 상관관계 분석(Correlation Map)

#08-1 PyCaret Initialization

❖ PyCaret 설정(Setup)

```
avc = setup(  
    data = ds,  
    target = "AveragePrice",  
    train_size = 0.8,  
    categorical_features = ['type', 'year', 'region', 'month'],  
    normalize = True,  
    normalize_method = 'robust',  
    verbose = 0,  
    preprocess = True,  
    session_id = 123  
)
```

- ❑ 타깃 변수
- ❑ 학습/테스트 비율
- ❑ 범주형 변수
- ❑ 정규화 방법
- ❑ 낮은 분산 변수 제거
- ❑ 난수 고정값

#08-1 PyCaret Initialization

❖ PyCaret 회귀 모델 목록

ID	Name	Reference
lr	Linear Regression	sklearn.linear_model._base.LinearRegression
lasso	Lasso Regression	sklearn.linear_model._coordinate_descent.Lasso
ridge	Ridge Regression	sklearn.linear_model._ridge.Ridge
en	Elastic Net	sklearn.linear_model._coordinate_descent.Elast...
lar	Least Angle Regression	sklearn.linear_model._least_angle.Lars
llar	Lasso Least Angle Regression	sklearn.linear_model._least_angle.LassoLars
omp	Orthogonal Matching Pursuit	sklearn.linear_model._omp.OrthogonalMatchingPu...
br	Bayesian Ridge	sklearn.linear_model._bayes.BayesianRidge
ard	Automatic Relevance Determination	sklearn.linear_model._bayes.ARDRegression
par	Passive Aggressive Regressor	sklearn.linear_model._passive_aggressive.Passi...
ransac	Random Sample Consensus	sklearn.linear_model._ransac.RANSACRegressor
tr	TheilSen Regressor	sklearn.linear_model._theil_sen.TheilSenRegressor
huber	Huber Regressor	sklearn.linear_model._huber.HuberRegressor
kr	Kernel Ridge	sklearn.kernel_ridge.KernelRidge
svm	Support Vector Regression	sklearn.svm._classes.SVR
knn	K Neighbors Regressor	sklearn.neighbors._regression.KNeighborsRegressor
dt	Decision Tree Regressor	sklearn.tree._classes.DecisionTreeRegressor
rf	Random Forest Regressor	sklearn.ensemble._forest.RandomForestRegressor
et	Extra Trees Regressor	sklearn.ensemble._forest.ExtraTreesRegressor
ada	AdaBoost Regressor	sklearn.ensemble._weight_boosting.AdaBoostRegr...
gbr	Gradient Boosting Regressor	sklearn.ensemble._gb.GradientBoostingRegressor
mlp	MLP Regressor	sklearn.neural_network._multilayer_perceptron....
lightgbm	Light Gradient Boosting Machine	lightgbm.sklearn.LGBMRegressor
dummy	Dummy Regressor	sklearn.dummy.DummyRegressor

❖ 데이터 전처리

- 날짜 포맷 변환
- 월 추출

```
ds.Date = pd.to_datetime(ds.Date)
ds['month'] = pd.DatetimeIndex(ds['Date']).month
```

#08-2 PyCaret Model Comparison

❖ PyCaret 회귀 모델 비교

□ 다양한 비교 기준

- MAE
- MSE
- RMSE
- R^2
- RMSLE (로그 스케일)
- MAPE (평균 절대 백분율)
- Time

❖ 성능 Top 3 모델

순위	모델 이름	R^2 Score
1	Extra Trees Regressor	0.9284
2	Random Forest Regressor	0.9073
3	LightGBM Regressor	0.8904

```
best_models = compare_models(sort='R2')
```

정렬 기준 R^2 Score

	Model	MAE	MSE	RMSE	R^2	RMSLE	MAPE
et	Extra Trees Regressor	0.0739	0.0117	0.1081	0.9278	0.0434	0.0545
rf	Random Forest Regressor	0.0801	0.0132	0.1146	0.9187	0.0460	0.0592
lightgbm	Light Gradient Boosting Machine	0.0986	0.0174	0.1320	0.8922	0.0535	0.0731
dt	Decision Tree Regressor	0.1115	0.0284	0.1685	0.8241	0.0673	0.0813
gbr	Gradient Boosting Regressor	0.1380	0.0341	0.1846	0.7894	0.0740	0.1024
knn	K Neighbors Regressor	0.1563	0.0466	0.2158	0.7122	0.0862	0.1153
ada	AdaBoost Regressor	0.1825	0.0523	0.2285	0.6771	0.0949	0.1443
ridge	Ridge Regression	0.1823	0.0583	0.2413	0.6401	0.0975	0.1373
br	Bayesian Ridge	0.1823	0.0583	0.2413	0.6401	0.0975	0.1373
lr	Linear Regression	0.1823	0.0583	0.2414	0.6401	0.0975	0.1374
huber	Huber Regressor	0.1858	0.0614	0.2477	0.6205	0.1001	0.1397
omp	Orthogonal Matching Pursuit	0.3166	0.1551	0.3937	0.0429	0.1625	0.2500
en	Elastic Net	0.3194	0.1584	0.3980	0.0220	0.1636	0.2523
lasso	Lasso Regression	0.3217	0.1602	0.4001	0.0114	0.1646	0.2542
llar	Lasso Least Angle Regression	0.3217	0.1602	0.4001	0.0114	0.1646	0.2542
dummy	Dummy Regressor	0.3243	0.1622	0.4027	-0.0013	0.1655	0.2561
par	Passive Aggressive Regressor	0.5822	3.3250	1.5922	-19.3304	0.2908	0.5038
lar	Least Angle Regression	10120062.8801	12707892102778052.0000	59322396.5113	-76460937197966352.0000	10.6155	8757715.2499

#08-2 PyCaret Model Comparison

❖ 모델 튜닝

❑ `tune_model(Model)`

❖ 모델 성능 비교

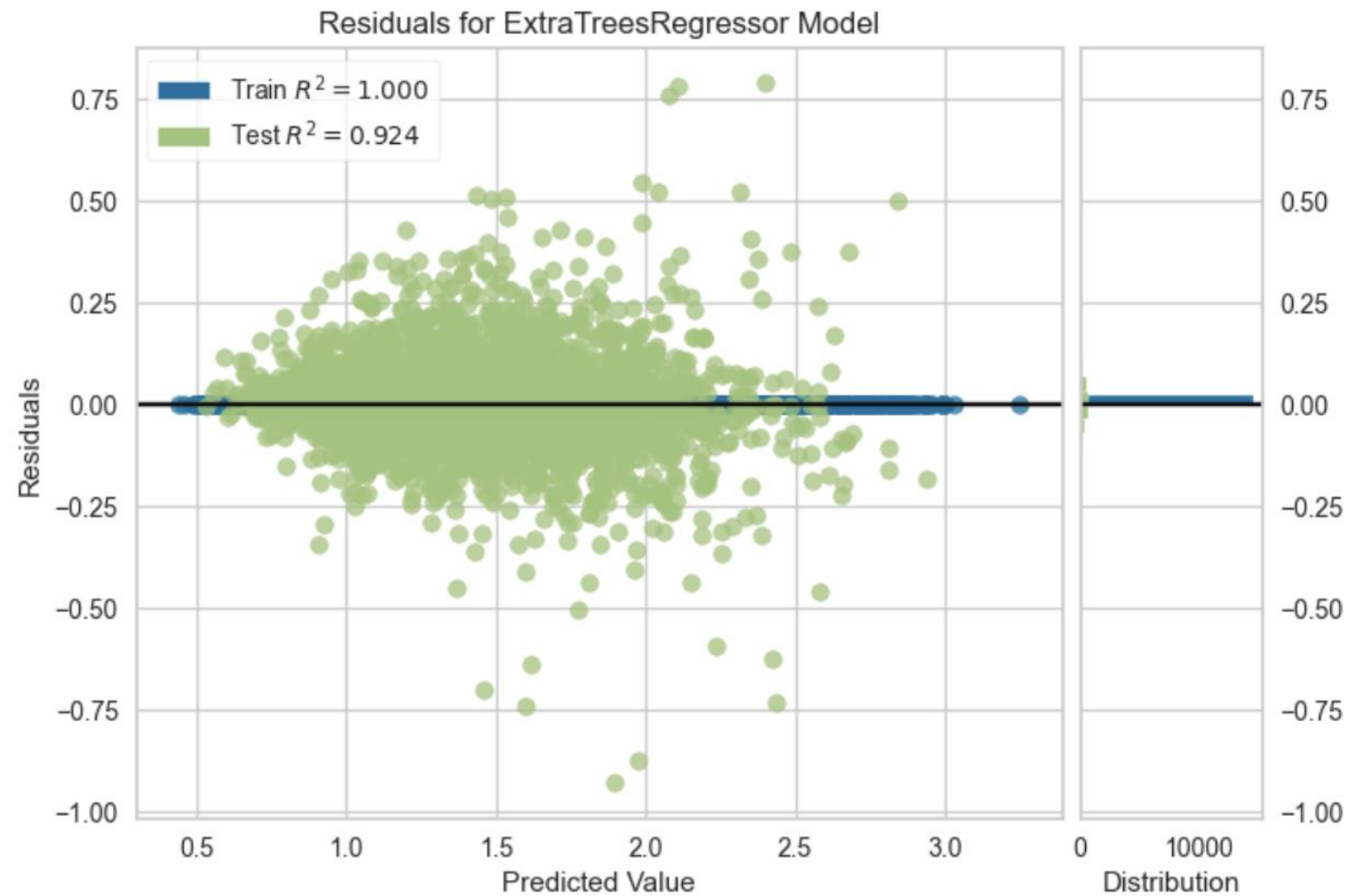
모델 이름	튜닝 여부	Train R²	Test R²
Extra Tree Regressor	✗ Not Tuned	1.000	0.925
	✓ Tuned	0.539	0.534
Random Forest Regressor	✗ Not Tuned	0.988	0.903
	✓ Tuned	0.647	0.629
Light Gradient Boosting	✗ Not Tuned	0.913	0.879
	✓ Tuned	0.870	0.823

- ❑ 모든 모델에서 튜닝 후 성능 감소
- ❑ Extra Tree Regressor가 가장 높은 정확도

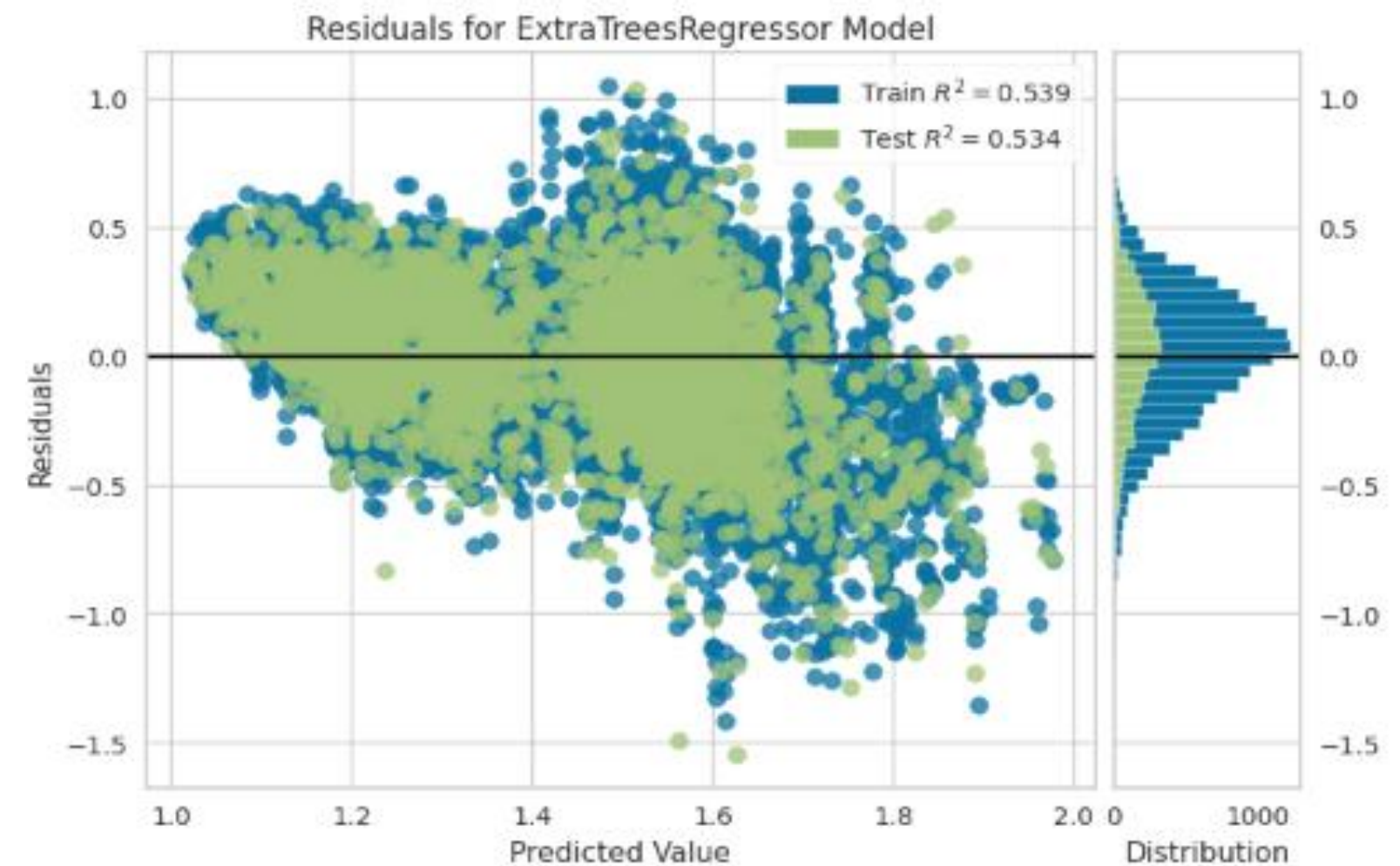
#09 Plots for Best Model(Extra Tree Regressor)

❖ 전차 그래프 (Residual Plot)

□ 튜닝 전



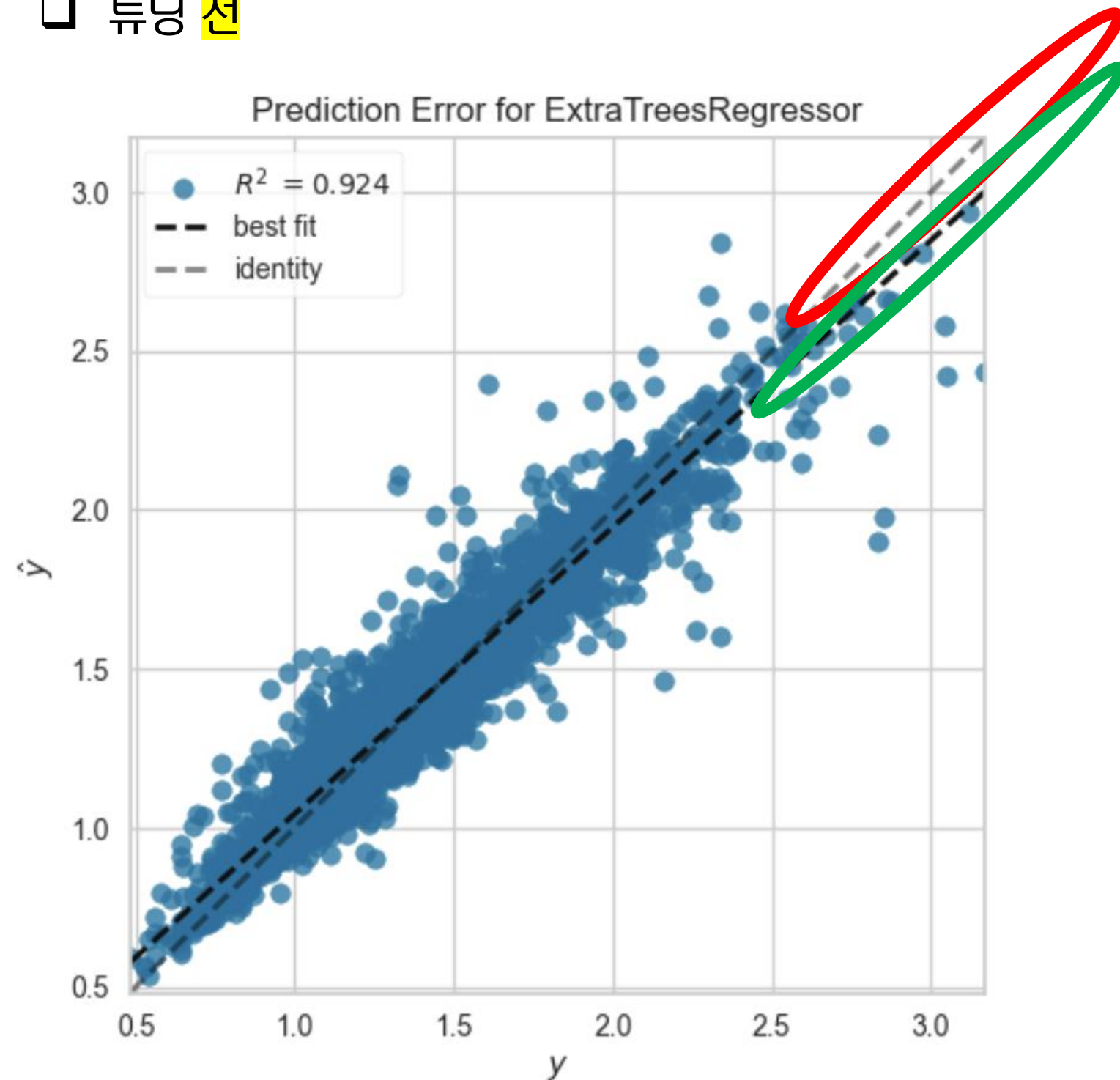
□ 튜닝 후



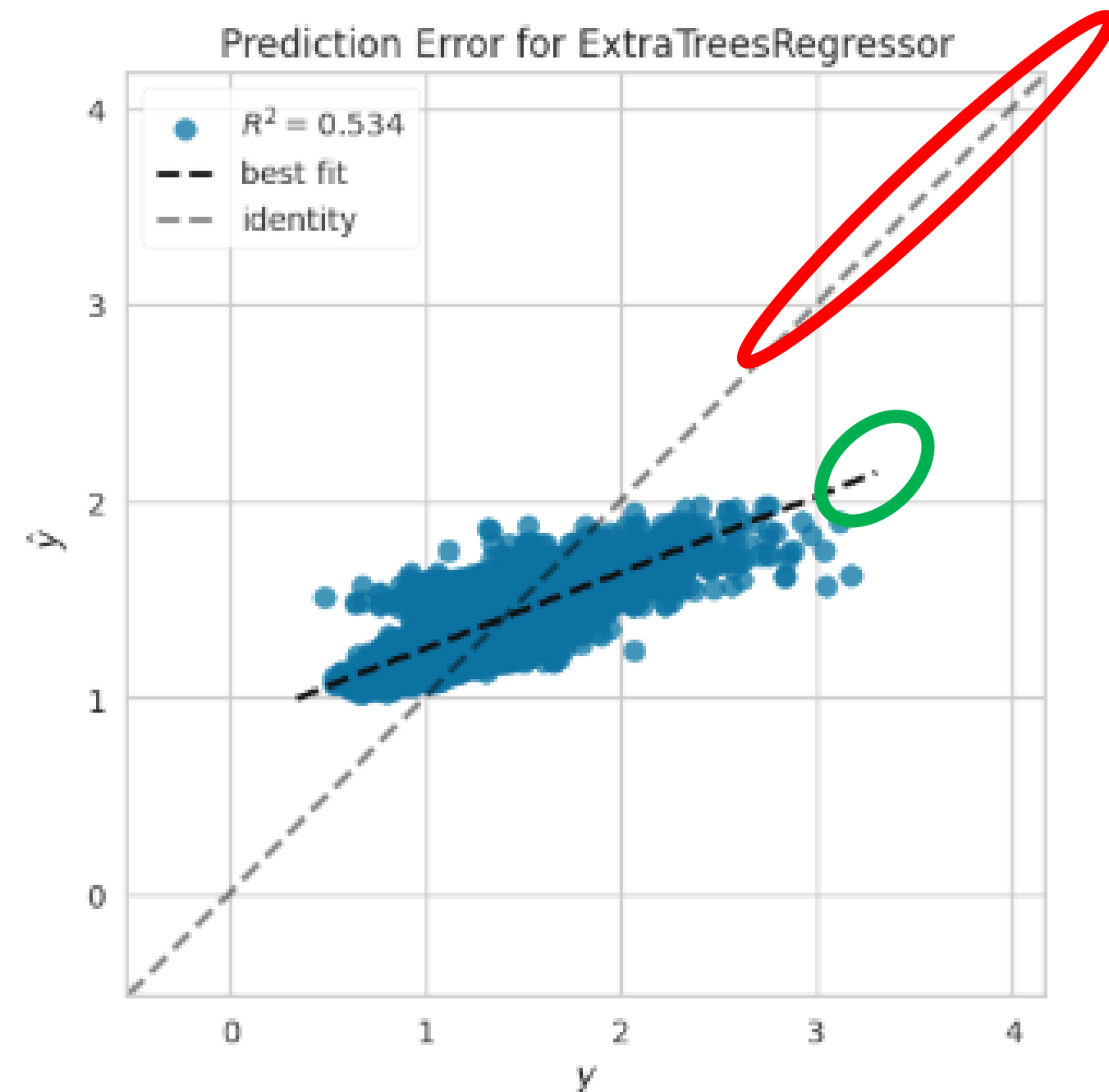
#09 Plots for Best Model(Extra Tree Regressor)

❖ 예측 오차 그래프 (Prediction Error Plot)

□ 튜닝 전



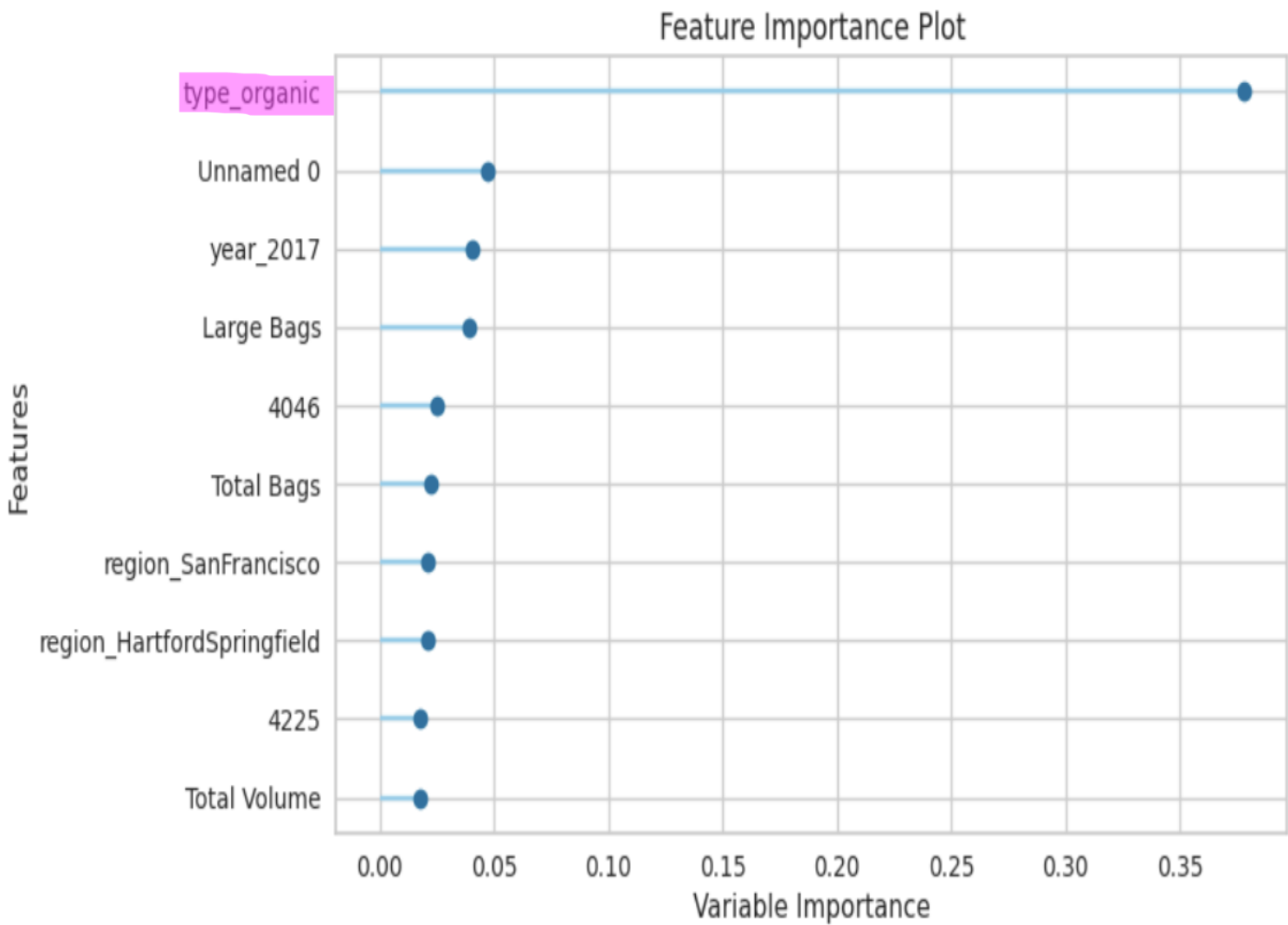
□ 튜닝 후



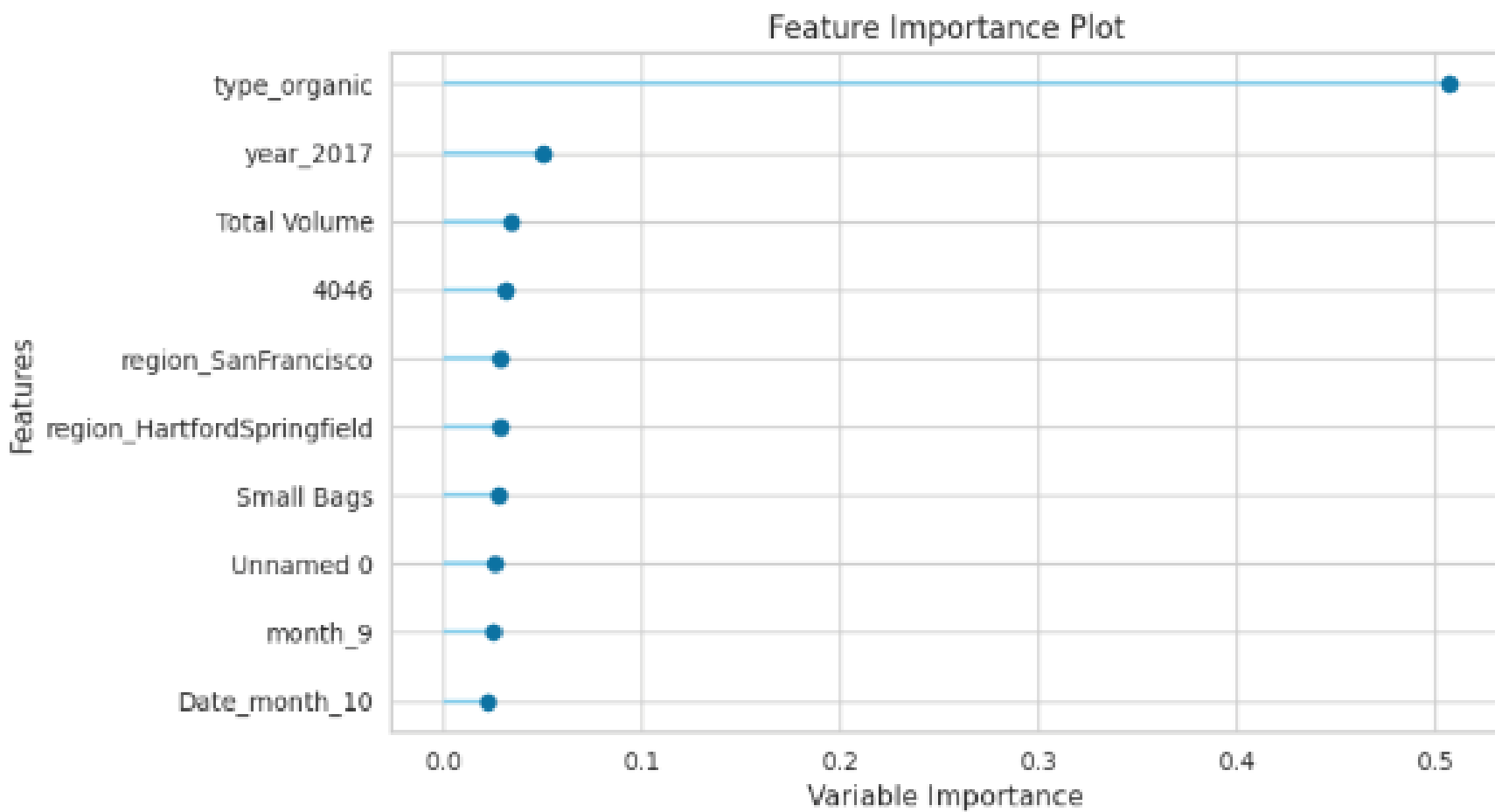
#09 Plots for Best Model(Extra Tree Regressor)

❖ 주요 변수 중요도 그래프(Feature Importance Plot)

□ 튜닝 전



□ 튜닝 후



#10 Create Models

❖ Random Forest Regressor Model

```
rf = create_model('rf')
```

	MAE	MSE	RMSE	R2	RMSLE	MAPE
Fold						
0	0.0813	0.0140	0.1183	0.9164	0.0466	0.0591
1	0.0800	0.0120	0.1096	0.9304	0.0446	0.0587
2	0.0791	0.0127	0.1129	0.9155	0.0461	0.0594
3	0.0791	0.0135	0.1161	0.9180	0.0459	0.0578
4	0.0766	0.0114	0.1066	0.9306	0.0426	0.0563
5	0.0765	0.0123	0.1107	0.9191	0.0448	0.0571
6	0.0802	0.0137	0.1173	0.9142	0.0467	0.0589
7	0.0835	0.0147	0.1213	0.9133	0.0478	0.0608
8	0.0835	0.0137	0.1171	0.9131	0.0477	0.0624
9	0.0809	0.0136	0.1165	0.9162	0.0474	0.0611
Mean	0.0801	0.0132	0.1146	0.9187	0.0460	0.0592
Std	0.0023	0.0010	0.0043	0.0062	0.0015	0.0018

❖ Light Gradient Boosting Model

```
lgbm = create_model('lightgbm')
```

	MAE	MSE	RMSE	R2	RMSLE	MAPE
Fold						
0	0.0988	0.0176	0.1327	0.8948	0.0528	0.0719
1	0.1005	0.0172	0.1310	0.9005	0.0531	0.0740
2	0.0990	0.0171	0.1309	0.8864	0.0542	0.0751
3	0.0943	0.0166	0.1287	0.8993	0.0516	0.0693
4	0.0936	0.0151	0.1229	0.9078	0.0495	0.0690
5	0.0966	0.0166	0.1289	0.8903	0.0526	0.0720
6	0.0989	0.0181	0.1345	0.8872	0.0541	0.0732
7	0.1024	0.0196	0.1399	0.8846	0.0557	0.0746
8	0.0994	0.0173	0.1315	0.8903	0.0541	0.0747
9	0.1019	0.0193	0.1388	0.8810	0.0568	0.0775
Mean	0.0986	0.0174	0.1320	0.8922	0.0535	0.0731
Std	0.0028	0.0012	0.0047	0.0079	0.0019	0.0025

#11 Predict Models

- ❖ PyCaret 의 `predict_model()` 함수 사용
 - ❑ 모델 튜닝 없이 테스트 데이터 예측
 - ❑ 각 모델이 예측한 결과값은 PyCaret에서 자동 생성
 - ❑ R^2 기준으로 평가한 모델 성능 순위:
 1. Extra Tree Regressor -> Test R^2 = 0.925
 2. Random Forest Regressor -> Test R^2 = 0.903
 3. Light GBM -> Test R^2 = 0.879

- ❑ Prediction using Extra Trees Regressor

Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
Extra Trees Regressor	0.0734	0.0122	0.1106	0.9244	0.0440	0.0540

- ❑ Prediction using Random Forest Regressor

Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
Random Forest Regressor	0.0798	0.0137	0.1172	0.9151	0.0466	0.0586

- ❑ Prediction using Light Gradient Boosting Machine

Model	MAE	MSE	RMSE	R2	RMSLE	MAPE
Light Gradient Boosting Machine	0.1018	0.0192	0.1387	0.8811	0.0552	0.0746

#12 Finalize Model

❖ 최종 모델 확정 및 저장

- 선택된 모델: Extra Tree Regressor
 - 실험된 모든 모델 중 가장 높은 정확도
 - 튜닝하지 않은 기본 모델

❖ 수행 작업:

- `finalize_model()` :
 - 전체 데이터로 모델을 재학습해 최종 버전 생성
- `predict_model()` :
 - 최종 모델을 사용해 테스트 데이터 예측 수행
- `plot_model(plot='parameter')` :
 - 하이퍼파라미터 시각화
- `save_model()` :
 - 모델을 .pkl 파일로 저장 -> 추후 배포 또는 사용 가능

Parameters	
bootstrap	False
ccp_alpha	0.0
criterion	squared_error
max_depth	None
max_features	1.0
max_leaf_nodes	None
max_samples	None
min_impurity_decrease	0.0
min_samples_leaf	1
min_samples_split	2
min_weight_fraction_leaf	0.0
monotonic_cst	None
n_estimators	100
n_jobs	-1
oob_score	False
random_state	123
verbose	0
warm_start	False

THANK YOU

